Finalna wersja specyfikacji

FlatShare

Grupa projektowa:
Jakub Rak, Bartosz Radomski, Sofiia Kuzmenko, Piotr Wójcik

Spis treści

1 Wstęp **5**
1.1 Cel i ogólna koncepcja systemu . . . 5
1.2 Zakres funkcjonalny i granice systemu . . . 5
1.3 Wysokopoziomowa struktura systemu . . . 5
1.4 Spójność pomiędzy wymaganiami, domeną i API . . . 5

2 Historie Użytkownika (User Stories) **6**
2.1 Aktor: Użytkownik (Wspólne) . . . 6
2.2 Aktor: Lokator . . . 6
2.3 Aktor: Właściciel Mieszkania . . . 6
2.4 Aktor: Administrator . . . 6
2.5 Kryteria akceptacji dla kluczowych historii użytkownika . . . 6
2.5.1 US-Sys1: Bezpieczne logowanie (JWT) . . . 7
2.5.2 US-W1: Wystawienie ogłoszenia . . . 7
2.5.3 US-L2: Zdefiniowanie preferencji i dopasowywanie . . . 7
2.5.4 US-L4/US-W4: Rezerwacja i akceptacja/odrzucenie . . . 7
2.5.5 US-L5: Płatność za rezerwację . . . 7
2.5.6 US-A1/US-A2: Moderacja i blokada użytkownika . . . 7

3 Diagramy Przypadków Użycia (Use Cases) **8**
3.1 Moduł Ogłoszeń . . . 8
3.2 Moduł Użytkownika . . . 9
3.3 Moduł Wynajmu i Płatności . . . 10
3.4 Moduł moderacji . . . 11

4 Szczegółowa specyfikacja (Deep Dive) **11**
4.1 Przypadek: Wystawienie ogłoszenia . . . 12
4.2 Przypadek: Rezerwacja pokoju (z płatnością) . . . 12
4.3 Przypadek: Blokada użytkownika (Moderacja) . . . 13

5 Diagramy Klas i Architektura (Class Diagrams) **14**
5.1 Model domenowy (encje i powiązania) . . . 14
5.2 Warstwa serwisów i repozytoriów (DIP/OCP) . . . 15
5.3 Moduły i komunikacja . . . 15
5.4 Kluczowe klasy i ich odpowiedzialność . . . 15
5.5 Wymiana informacji między modułami (Inter-module communication) . . . 16

6 Diagramy stanów i aktywności **16**
6.1 Cel i zakres . . . 16
6.2 Diagramy stanów kluczowych obiektów biznesowych . . . 17
6.2.1 Obiekt: `User` (konto użytkownika) . . . 17
6.2.2 Obiekt: `Listing` (ogłoszenie) . . . 18
6.2.3 Obiekt: `Booking` (rezerwacja) . . . 19
6.2.4 Obiekt: `Payment` (płatność) . . . 20
6.2.5 Obiekt: `ViolationReport` (zgłoszenie naruszenia / sprawa moderacyjna) . . . 21
6.3 Diagramy aktywności kluczowych funkcjonalności . . . 22
6.3.1 Funkcjonalność: Wystawienie ogłoszenia (`Create Listing`) . . . 22
6.3.2 Funkcjonalność: Rezerwacja pokoju z płatnością (`Book a Room`) . . . 24
6.3.3 Funkcjonalność: Blokada użytkownika (`Ban User`) . . . 25

7 Założenia wspólne **26**
7.1 Konwencje zasobów REST i wersjonowanie . . . 26
7.2 Sytuacje wyścigu . . . 26
7.3 Wspólny format odpowiedzi błędów . . . 26
7.4 Słownik typów danych wykorzystywanych w komunikatach . . . 27

8 Wspólny mechanizm autoryzacji (JWT) **27**
8.1 Zakres stosowania . . . 27
8.2 Wymagane nagłówki HTTP . . . 27
8.3 Proces walidacji tokenu . . . 27
8.4 Obsługa błędów autoryzacji . . . 28
8.5 Uwagi projektowe . . . 28
8.6 Diagram sekwencji – walidacja tokenu JWT . . . 28

9 Rejestracja **29**
9.1 Opis endpointu . . . 29
9.2 Przykładowe żądanie . . . 29
9.3 Przykładowa odpowiedź — sukces . . . 30
9.4 Obsługa błędów . . . 30

10 Pobieranie danych użytkownika **30**
10.1 Opis endpointu . . . 30
10.2 Przykładowe żądanie . . . 31
10.3 Przykładowa odpowiedź — sukces . . . 31
10.4 Obsługa błędów . . . 31

11 Logowanie **31**
11.1 Opis endpointu . . . 31
11.2 Przykładowe żądanie . . . 32
11.3 Przykładowa odpowiedź — sukces . . . 32
11.4 Obsługa błędów . . . 32
11.5 Diagram sekwencji procesu logowania . . . 32
11.6 Uwagi projektowe . . . 33

12 Odświeżanie sesji **33**
12.1 Opis endpointu . . . 33
12.2 Przykładowe żądanie . . . 33
12.3 Przykładowa odpowiedź — sukces . . . 34
12.4 Obsługa błędów . . . 34

13 Pobieranie informacji o sesji **34**
13.1 Opis endpointu . . . 34
13.2 Przykładowe żądanie . . . 34
13.3 Przykładowa odpowiedź — sukces . . . 35
13.4 Obsługa błędów . . . 35
13.5 Reset hasła . . . 35
13.5.1 Etap 1: żądanie resetu . . . 35
13.5.2 Etap 2: potwierdzenie resetu . . . 35

14 Zarządzanie preferencjami **36**
14.1 Opis endpointu - ustawianie preferencji . . . 36
14.2 Opis endpointu - pobieranie preferencji . . . 36
14.3 Uwagi projektowe . . . 37

15 Wyszukiwanie pasujących mieszkań **37**
15.1 Opis endpointu 37
15.2 Parametry zapytania 37
15.3 Przykładowe żądanie 38
15.4 Przykładowa odpowiedź — sukces 38
15.5 Obsługa sytuacji brzegowych 38
15.6 Diagram sekwencji dopasowywania 39
15.7 Uwagi projektowe 39

16 Wystawienie ogłoszenia i zarządzanie nim **39**
16.1 Opis endpointu 40
16.2 Przykładowe żądanie 40
16.3 Przykładowa odpowiedź — sukces 40
16.4 Obsługa błędów 41
16.5 Diagram sekwencji wystawienia ogłoszenia 43
16.6 Uwagi projektowe 43

17 Zarządzanie zdjęciami ogłoszeń **44**

18 Wynajem pokoju **45**
18.1 Opis endpointu 45
18.2 Przykładowe żądanie 45
18.3 Przykładowa odpowiedź — sukces 46
18.4 Obsługa błędów 46
18.5 Diagram sekwencji wynajmu pokoju 46
18.6 Uwagi projektowe 46

19 Moderacja i zgłoszenia naruszeń **47**
19.1 Zgłoszenie naruszenia przez użytkownika 47
19.2 Panel Administratora — lista zgłoszeń 48
19.3 Blokada użytkownika 48

1 Wstęp

Dokument zawiera specyfikację wymagań funkcjonalnych oraz projekt architektury systemu. Celem systemu jest połączenie właścicieli mieszkań z potencjalnymi lokatorami poprzez mechanizmy dopasowania preferencji oraz bezpośrednią komunikację. System jest podzielony na niezależne moduły, które komunikują się poprzez warstwę serwisową.

1.1 Cel i ogólna koncepcja systemu

FlatShare odpowiada na typowy problem rynku najmu pokoi: duża liczba ogłoszeń, brak ustandaryzowanej informacji o preferencjach domowników oraz konieczność wielokrotnej, powtarzalnej komunikacji przed podjęciem decyzji o wynajmie. System ma zmniejszyć „tarcie" po obu stronach procesu:

- **Lokator** szybciej znajduje ogłoszenia pasujące do jego potrzeb (filtry + dopasowywanie) oraz ma bezpośredni kanał kontaktu z właścicielem.
- **Właściciel** publikuje ogłoszenia w spójnej formie, kontroluje dostępność i może akceptować rezerwacje.
- **Administrator** utrzymuje jakość platformy poprzez obsługę zgłoszeń naruszeń i blokowanie kont łamiących regulamin.

1.2 Zakres funkcjonalny i granice systemu

Zakres systemu obejmuje trzy główne obszary: (1) ogłoszenia i wyszukiwanie, (2) komunikację i profilowanie użytkowników, (3) rezerwacje oraz płatności. W ramach projektu zakłada się, że:

- System obsługuje **wynajem pokoju** w ramach mieszkania (model `Apartment/Room`).
- Dane o płatnościach są przetwarzane **pośrednio** — FlatShare inicjuje płatność i utrzymuje jej status, natomiast właściwe obciążenie realizuje zewnętrzna bramka płatnicza.
- Zdjęcia ogłoszeń są przechowywane w zewnętrznym magazynie plików (np. obiektowe *file storage*), a system przechowuje referencje/URL-e.

Poza zakresem (w tej iteracji) pozostają m.in.: generowanie umów najmu, rozbudowane rozliczenia cykliczne (czynsz co miesiąc), zaawansowany KYC użytkowników oraz wielowalutowe rozliczenia z przewalutowaniem.

1.3 Wysokopoziomowa struktura systemu

System jest projektowany jako aplikacja z rozdzieleniem na warstwy: **kontrolery REST** (wejście HTTP), **serwisy aplikacyjne** (logika przypadków użycia) oraz **repozytoria** (trwałość danych). Dodatkowo wydzielono porty integracyjne (np. bramka płatności, powiadomienia) w celu utrzymania niskiego sprzężenia pomiędzy logiką domenową a infrastrukturą. Taki podział jest widoczny na diagramach klas i modułów w dalszej części dokumentu.

1.4 Spójność pomiędzy wymaganiami, domeną i API

W dalszych rozdziałach wymagania (historie użytkownika) są mapowane na przypadki użycia, a następnie na obiekty domenowe (np. `Listing`, `Booking`, `Payment`) oraz endpointy API. Dzięki temu jedna funkcjonalność ma jedno „źródło prawdy" w modelu domenowym, a diagramy sekwencji i przykładowe komunikaty API odzwierciedlają te same reguły biznesowe (np. rezerwacja przechodzi do `CONFIRMED` dopiero po sukcesie płatności).

2 Historie Użytkownika (User Stories)

Poniżej przedstawiono wymagania biznesowe zagregowane według aktorów systemu.

2.1 Aktor: Użytkownik (Wspólne)

- **US-Sys1:** Jako Użytkownik, chcę bezpiecznie logować się do systemu, aby uzyskać dostęp do swojego konta.
- **US-Sys2:** Jako Użytkownik, chcę mieć możliwość resetu hasła, aby odzyskać dostęp do konta w przypadku jego zapomnienia.

2.2 Aktor: Lokator

- **US-L1:** Jako Lokator, chcę przeglądać i filtrować ogłoszenia (wg ceny, lokalizacji), aby szybko znaleźć pokój spełniający moje kryteria.
- **US-L2:** Jako Lokator, chcę zdefiniować swoje preferencje (np. zwierzęta, palenie), aby system mógł podpowiadać mi najlepiej dopasowane oferty.
- **US-L4:** Jako Lokator, chcę wysłać prośbę o rezerwację pokoju, aby sfinalizować proces wynajmu.
- **US-L5:** Jako Lokator, chcę opłacić rezerwację przez zewnętrzny system płatności, aby potwierdzić wynajem.

2.3 Aktor: Właściciel Mieszkania

- **US-W1:** Jako Właściciel, chcę wystawić ogłoszenie ze zdjęciami i opisem, aby dotrzeć do potencjalnych najemców.
- **US-W2:** Jako Właściciel, chcę określić preferowany profil lokatora (np. student, osoba pracująca), aby uniknąć nieporozumień.
- **US-W3:** Jako Właściciel, chcę zarządzać dostępnością ogłoszenia (publikować/ukrywać/archiwizować), gdy mieszkanie zostanie wynajęte.
- **US-W4:** Jako Właściciel, chcę zaakceptować lub odrzucić rezerwację, aby kontrolować kto wynajmuje pokój.

2.4 Aktor: Administrator

- **US-A1:** Jako Admin, chcę mieć możliwość blokowania użytkowników łamiących regulamin, aby utrzymać porządek w serwisie.
- **US-A2:** Jako Admin, chcę przeglądać zgłoszone naruszenia w panelu administracyjnym, aby szybko reagować na nadużycia.

2.5 Kryteria akceptacji dla kluczowych historii użytkownika

Poniższe kryteria akceptacji stanowią „rewersy” historii użytkownika i są wykorzystywane jako podstawa do weryfikacji implementacji. W kryteriach celowo pojawiają się terminy z domeny (`Listing`, `Booking`, `Payment`, `MessageThread`), aby zachować spójność z dalszym opisem modeli, stanów oraz komunikacji API.

2.5.1 US-Sys1: Bezpieczne logowanie (JWT)

- **AC1:** Dla aktywnego użytkownika z poprawnymi danymi uwierzytelniającymi system zwraca `200 OK` oraz token JWT zawierający identyfikator użytkownika i role.
- **AC2:** Dla błędnego loginu lub hasła system zwraca `401 Unauthorized` bez ujawniania informacji o tym, czy konto istnieje.
- **AC3:** Token jest wymagany dla endpointów chronionych, a brak tokenu skutkuje `401 Unauthorized`.

2.5.2 US-W1: Wystawienie ogłoszenia

- **AC1:** Właściciel z rolą `LANDLORD` może utworzyć ogłoszenie z wymaganymi danymi (tytuł, opis, cena, lokalizacja, dostępność), a system przypisuje je do jego konta.
- **AC2:** System odrzuca niepoprawne dane (np. cena ≤ 0) odpowiedzią `400 Bad Request` oraz zwraca listę błędów pól.

2.5.3 US-L2: Zdefiniowanie preferencji i dopasowywanie

- **AC1:** Lokator może zapisać zestaw preferencji (np. palenie, zwierzęta, budżet), a system przechowuje je w kontekście roli `TenantRole`.
- **AC2:** Endpoint dopasowań zwraca ogłoszenia posortowane malejąco po *match score*.
- **AC3:** Jeśli preferencje nie są ustawione, system zwraca pustą listę (lub zgodnie z polityką — `404 Not Found`).

2.5.4 US-L4/US-W4: Rezerwacja i akceptacja/odrzucenie

- **AC1:** Lokator może złożyć prośbę o rezerwację tylko dla ogłoszenia w stanie `ACTIVE` i dostępnego terminu.
- **AC2:** Właściciel może zaakceptować lub odrzucić rezerwację; decyzja zmienia stan `Booking` odpowiednio na `PENDING_PAYMENT` lub `REJECTED`.

2.5.5 US-L5: Płatność za rezerwację

- **AC1:** System inicjuje płatność w bramce zewnętrznej i zapisuje obiekt `Payment` w stanie `Initiated/Redirected`.
- **AC2:** Po potwierdzeniu płatności status `Payment` zmienia się na `Succeeded`, a `Booking` na `CONFIRMED`.
- **AC3:** Nieudana płatność skutkuje `Failed` oraz mapowaniem na `PAYMENT_FAILED` po stronie `Booking`; użytkownik może ponowić próbę.

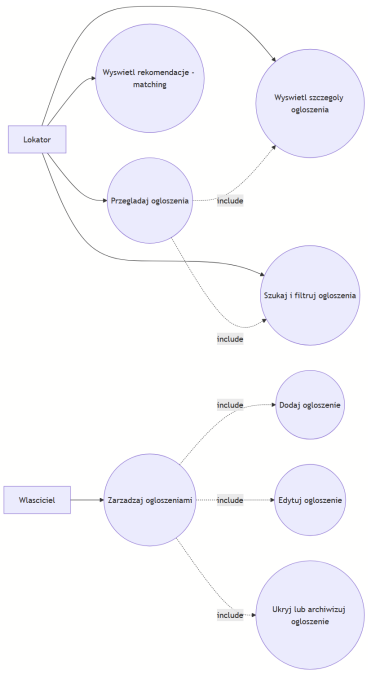
2.5.6 US-A1/US-A2: Moderacja i blokada użytkownika

- **AC1:** Administrator widzi listę zgłoszeń naruszeń wraz ze statusem sprawy (`Open/UnderReview/...`).
- **AC2:** Zablokowanie użytkownika ustawia `User.status = BLOCKED` oraz ukrywa jego aktywne ogłoszenia.
- **AC3:** Zablokowany użytkownik nie może tworzyć nowych ogłoszeń ani wysyłać wiadomości; próba skutkuje `403 Forbidden`.

3 Diagramy Przypadków Użycia (Use Cases)

Funkcjonalności zostały podzielone na logiczne moduły, co ułatwia analizę systemu. Diagramy przypadków użycia stanowią ilustrację do wstępów poszczególnych grup funkcjonalności.

3.1 Moduł Ogłoszeń



Rysunek 1: Diagram przypadków użycia - Moduł Ogłoszeń

Opis modułu. Moduł Ogłoszeń koncentruje się na zarządzaniu zasobem `Listing` oraz jego prezentacji po stronie Lokatora. Jak pokazuje rysunek, Lokator może przeglądać ogłoszenia, korzystać z filtrów oraz przechodzić do szczegółów wybranego ogłoszenia. Równolegle system udostępnia widok rekomendacji (*matching*), który wykorzystuje preferencje Lokatora do wyliczenia dopasowania.

Główne funkcje po stronie Lokatora:

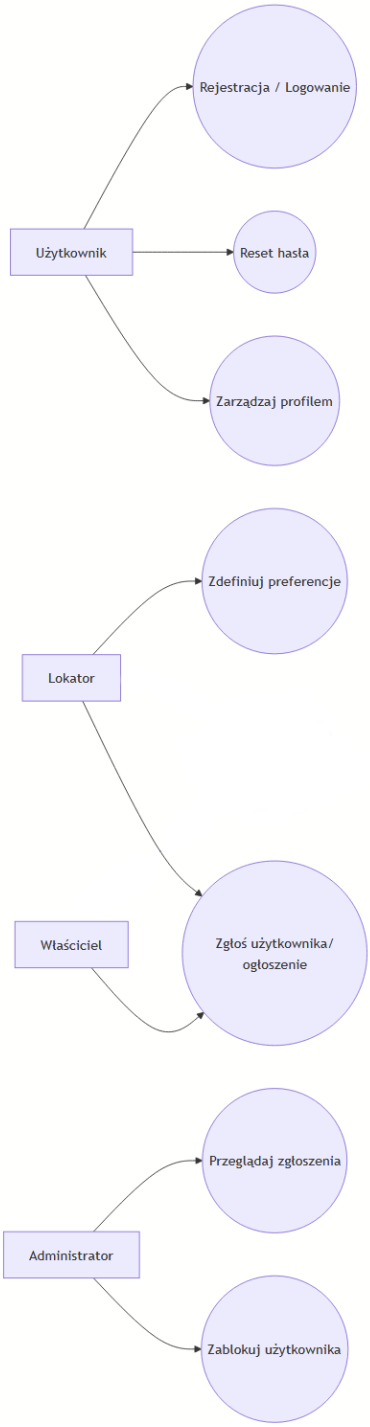
- *Przeglądaj ogłoszenia* — podstawowa lista ofert, stanowiąca punkt startowy dla dalszych akcji.
- *Szukaj / filtruj ogłoszenia* — zawężanie wyników m.in. po cenie i lokalizacji (realizacja US-L1).
- *Wyświetl szczegóły ogłoszenia* — wejście w kontekst konkretnego `Listing`, niezbędne m.in. do wiadomości i rezerwacji.
- *Wyświetl rekomendacje / matching* — wariant listy ofert posortowanej po *match score* (powiązanie z US-L2).

Główne funkcje po stronie Właściciela:

- *Dodaj ogłoszenie* oraz *Edytuj ogłoszenie* — przygotowanie kompletnej oferty do publikacji (US-W1).
- *Ukryj lub archiwizuj ogłoszenie* — kontrola dostępności i zakończenie cyklu życia ogłoszenia (US-W3), spójne ze stanami `Hidden/Archived` opisanymi później.

W kolejnych rozdziałach diagramy stanów i aktywności doprecyzowują, kiedy ogłoszenie jest widoczne (`ACTIVE`) i jakie akcje są dozwolone w poszczególnych stanach.

3.2 Moduł Użytkownika

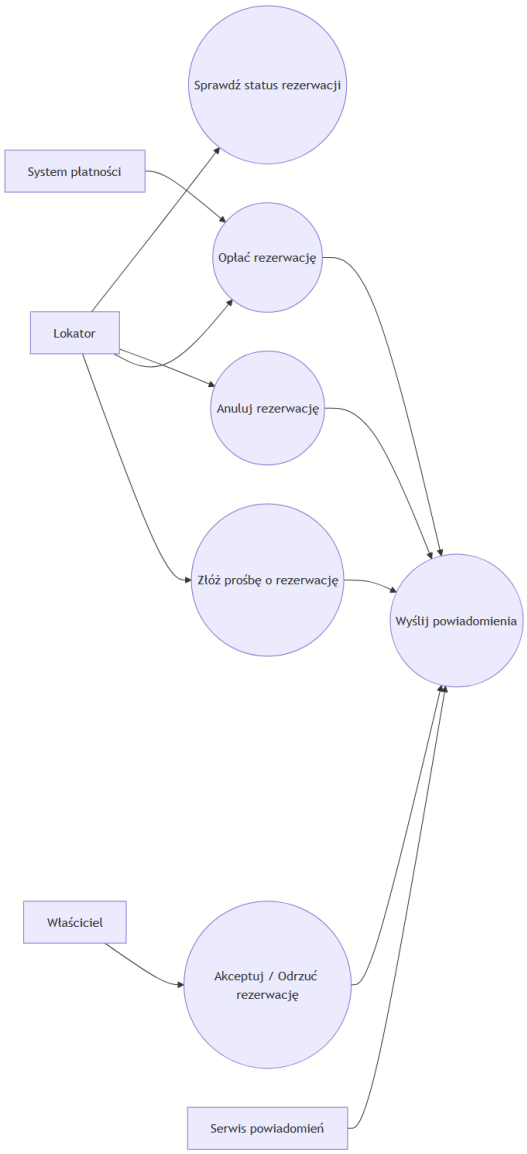


Rysunek 2: Diagram przypadków użycia - Moduł Użytkownika i Komunikacji

Opis modułu. Moduł Użytkownika łączy funkcje związane z kontem (`User`), rolami oraz wymianą wiadomości pomiędzy Lokatorem i Właścicielem. Z perspektywy bezpieczeństwa to właśnie w tym obszarze powstaje token JWT wykorzystywany później przez pozostałe moduły.

Zarządzanie kontem: rejestracja/logowanie, reset hasła oraz zarządzanie profilem stanowią funkcje wspólne, niezależne od roli użytkownika. Profil obejmuje dane kontaktowe i ustawienia, natomiast `User.status` determinuje, czy użytkownik może korzystać z systemu (np. `BLOCKED` po moderacji).
Preferencje i dopasowanie: Lokator może zdefiniować preferencje, które są wykorzystywane przez moduł dopasowań do wyliczenia rekomendacji. Ważne jest rozdzielenie danych „kontowych" od danych roli — preferencje są przechowywane w `TenantRole`, co jest spójne z modelem domenowym.
Zgłoszenia: Zarówno Lokator, jak i Właściciel mogą zgłosić użytkownika lub ogłoszenie, co inicjuje proces moderacyjny obsługiwany przez Administratora.

3.3 Moduł Wynajmu i Płatności

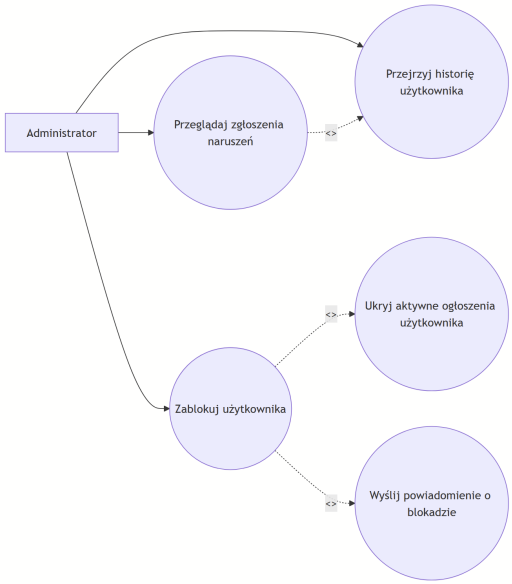


Rysunek 3: Diagram przypadków użycia - Moduł Wynajmu/Rezerwacji i Płatności (realizacja US-L4, US-L5, US-W4, US-W5)

Opis modułu. Moduł Wynajmu i Płatności obejmuje pełny przepływ rezerwacji pokoju: od złożenia prośby, przez decyzję Właściciela, aż po opłacenie i potwierdzenie. W module występują

również integracje zewnętrzne: **System Płatności** (realizacja obciążenia) oraz **Serwis Powiadomień** (informowanie stron o zdarzeniach).
Rezerwacja jako obiekt biznesowy. Kluczowym obiektem jest `Booking` (w API opisywany jako „wynajem"), który przechowuje status procesu. Wariant standardowy wymaga akceptacji Właściciela (`PENDING_APPROVAL`).
Płatność i powiadomienia. Płatność jest śledzona w osobnym obiekcie `Payment` i wpływa na stan rezerwacji. W praktyce moduł wysyła powiadomienia po kluczowych zdarzeniach: utworzenie rezerwacji, akceptacja/odrzucenie, sukces/porażka płatności oraz anulowanie. Dzięki temu obie strony mają spójny obraz postępu procesu.

3.4 Moduł moderacji



Rysunek 4: Moderacja — przypadki użycia

Opis modułu. Moduł moderacji odpowiada za utrzymanie jakości platformy oraz reagowanie na nadużycia. Proces zaczyna się od zgłoszenia naruszenia (obiekt `ViolationReport`), a następnie przechodzi przez analizę Administratora i kończy się decyzją: zamknięcie bez akcji lub podjęcie działań (np. blokada konta).
Konsekwencje blokady. Zablokowanie użytkownika jest decyzją systemową o skutkach wielomodułowych:

- `User.status` przechodzi do `BLOCKED`, co ogranicza dostęp do endpointów chronionych dla danego użytkownika,
- aktywne ogłoszenia są ukrywane (`Listing` → `HiddenByModeration` lub równoważny stan „ukryty"),
- system wysyła powiadomienie o blokadzie, aby zapewnić transparentność procesu.

Dalsze diagramy stanów i aktywności pokazują, jak te konsekwencje propagują się do cyklu życia konta i ogłoszeń.

4 Szczegółowa specyfikacja (Deep Dive)

Szczegółowy opis kluczowych przypadków użycia wg szablonu Cockburna (główne scenariusze oraz kluczowe alternatywy).

4.1 Przypadek: Wystawienie ogłoszenia

Nazwa przypadku	**Wystawienie ogłoszenia (Create Listing)**
Aktor główny	Właściciel Mieszkania
Aktorzy pomocniczy	System Przechowywania Plików (File Storage), (opcjonalnie) System Płatności
Warunki wstępne	Użytkownik zalogowany, posiada rolę Właściciela.
Główny scenariusz	1. Właściciel wybiera opcję "Dodaj ogłoszenie". 2. System prosi o dane (tytuł, opis, cena, lokalizacja) oraz zdjęcia. 3. Właściciel wprowadza dane i dodaje zdjęcia. 4. System waliduje dane (np. cena > 0, wymagane pola). 5. System zapisuje ogłoszenie (status: *DRAFT* lub *UNDER_REVIEW* zgodnie z polityką). 6. (Jeśli dotyczy) System publikuje ogłoszenie (status: *ACTIVE*).
Scenariusze alternatywne	**4a. Błąd walidacji:** System wyświetla błędy i prosi o poprawę. **5a. Limit darmowych publikacji:** Jeśli użytkownik przekroczył limit darmowych ogłoszeń, system przechodzi do płatności i dopiero po sukcesie umożliwia publikację.

4.2 Przypadek: Rezerwacja pokoju (z płatnością)

Nazwa przypadku	**Rezerwacja pokoju (Book a Room)**
Aktor główny	Lokator
Aktorzy pomocniczy	Właściciel, System Płatności (zewnętrzny), Serwis Powiadomień
Warunki wstępne	Lokator zalogowany, ogłoszenie ma status ACTIVE, pokój dostępny dla wybranego okresu.
Główny scenariusz	1. Lokator na stronie ogłoszenia klika "Rezerwuj". 2. System pokazuje podsumowanie kosztów, regulamin i prosi o potwierdzenie. 3. Lokator potwierdza chęć rezerwacji. 4. System tworzy rezerwację ze statusem `PENDING_APPROVAL` i wysyła powiadomienie do Właściciela. 5. Właściciel akceptuje rezerwację. System zmienia status na `PENDING_PAYMENT`, tymczasowo blokuje termin (uruchamia licznik czasu) i generuje link do płatności. 6. System przekierowuje Lokatora do Systemu Płatności. 7. System Płatności potwierdza płatność. System zmienia status rezerwacji na `CONFIRMED` (blokada terminu staje się stała). 8. Serwis Powiadomień wysyła potwierdzenia do Lokatora i Właściciela.

Scenariusze alternatywne	**5b. Brak reakcji/Timeout płatności:** Właściciel nie odpowiada lub Lokator nie opłaci rezerwacji w wyznaczonym czasie (np. 15 min). System ustawia status `EXPIRED`, zwalnia blokadę terminu i powiadamia Lokatora. **7a. Błąd płatności:** Płatność nieudana. System ustawia status `PAYMENT_FAILED` i umożliwia ponowienie lub anulowanie.

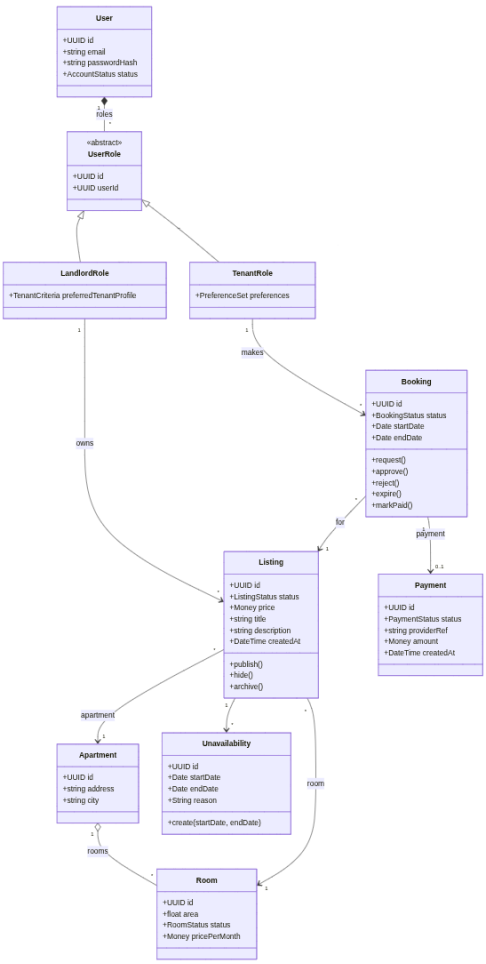
4.3 Przypadek: Blokada użytkownika (Moderacja)

Nazwa przypadku	**Blokada użytkownika (Ban User)**
Aktor główny	Administrator
Warunki wstępne	Admin zalogowany do panelu administracyjnego.
Główny scenariusz	1. Admin przegląda listę zgłoszeń naruszeń. 2. Admin analizuje historię zgłoszonego użytkownika. 3. Admin wybiera opcję "Zablokuj konto" i podaje powód. 4. System zmienia status użytkownika na *BLOCKED*. 5. Wszystkie aktywne ogłoszenia użytkownika są ukrywane. 6. System wyszukuje aktywne rezerwacje (CONFIRMED) u zablokowanego właściciela, zmienia ich status na CANCELLED/REFUNDED i zleca zwrot środków. 7. Serwis Powiadomień wysyła wiadomość email z informacją o blokadzie oraz o anulowaniu rezerwacji.
Scenariusze alternatywne	**3a. Odrzucenie zgłoszenia:** Admin uznaje zgłoszenie za bezzasadne i zamyka sprawę bez blokady.

5 Diagramy Klas i Architektura (Class Diagrams)

Poniższe diagramy prezentują strukturę klas zaprojektowaną w celu realizacji funkcjonalności zdefiniowanych w diagramach przypadków użycia. Projekt rozdziela modele danych (encje domenowe) od logiki (serwisy), zgodnie z zasadami SOLID.

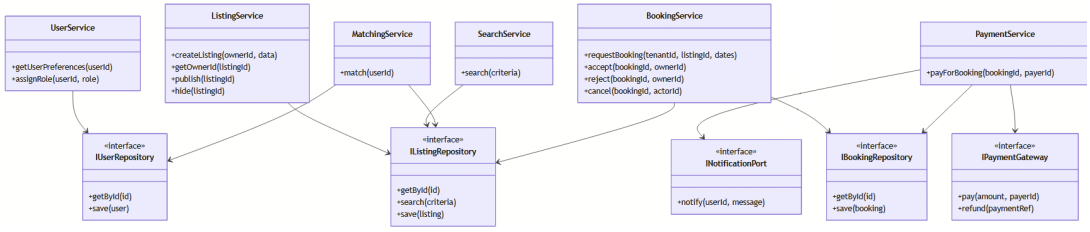
5.1 Model domenowy (encje i powiązania)



Rysunek 5: Diagram klas - model domenowy (User + Role, Listing, Apartment/Room, Booking, Payment)

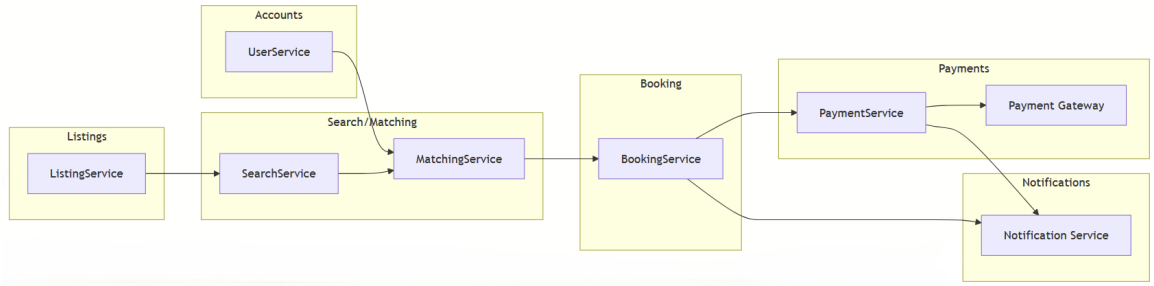
Uwaga projektowa (spójność domeny). Rezerwacja (`Booking`) jest zawsze tworzona przez Lokatora (rola `TenantRole`) i dotyczy konkretnego ogłoszenia (`Listing`). Z tego powodu w modelu domenowym `Booking` posiada powiązania do `TenantRole` oraz `Listing`.

5.2 Warstwa serwisów i repozytoriów (DIP/OCP)



Rysunek 6: Diagram klas - serwisy, repozytoria i porty integracyjne (płatności, powiadomienia)

5.3 Moduły i komunikacja



Rysunek 7: Podział na moduły i kanały komunikacji (inter-module communication)

Interpretacja diagramu modułów. Moduły na rysunku należy rozumieć jako **logiczne komponenty** w obrębie jednej aplikacji serwerowej (nie jako osobne mikroserwisy). Dzięki temu komunikacja pomiędzy modułami realizowana jest przez jawne interfejsy serwisów i porty integracyjne, co ogranicza sprzężenie i ułatwia testowanie.
Kanały komunikacji:

- **Wywołania synchroniczne (in-process)** pomiędzy modułami, np.
- **Integracje zewnętrzne** poprzez porty, np. `IPaymentGateway` (płatności) oraz `INotificationPort` (powiadomienia).

Warstwy aplikacji a moduły. Każdy moduł posiada analogiczną strukturę: kontrolery REST (wejście), serwisy aplikacyjne (logika przypadków użycia) oraz repozytoria (dostęp do danych). Wspólny model domenowy jest wykorzystywany w serwisach, natomiast „zewnętrzny świat" (HTTP, bazy danych, bramki) jest izolowany w warstwie infrastruktury. Takie podejście jest spójne z przedstawioną dalej warstwą serwisów i repozytoriów (DIP/OCP).

5.4 Kluczowe klasy i ich odpowiedzialność

Zgodnie z wymaganiami projektowymi, system opiera się na obiektach domenowych oraz serwisach zarządzających logiką.

1. **Użytkownicy i Role (Kompozycja):**
   - Zamiast dziedziczenia ról od `User`, zastosowano kompozycję: `User` posiada przypisane obiekty ról (np. `TenantRole`, `LandlordRole`).
   - `User` przechowuje dane uwierzytelniające oraz status konta.

- Role definiują uprawnienia i kontekst danych: preferencje należą do roli Lokatora, a zarządzanie mieszkaniami/ogłoszeniami do roli Właściciela.

2. **Ogłoszenia (Listing) oraz struktura lokalu:**
   - `Listing` reprezentuje ogłoszenie powiązane z `Apartment` i `Room`.
   - **Status ogłoszenia:** `Listing.status` (np. *ACTIVE, HIDDEN, ARCHIVED*) pozwala zarządzać dostępnością i publikacją.
3. **Zarządzanie dostępnością (Unavailability):**
   - Wprowadzono encję `Unavailability` powiązaną relacją jeden-do-wielu z `Listing`.
   - Pozwala ona Właścicielowi na wyłączenie konkretnych zakresów dat z wynajmu (np. remont, użytek własny) bez konieczności usuwania ogłoszenia czy tworzenia fikcyjnych rezerwacji.
4. **Wynajem/Rezerwacja i Płatność:**
   - `Booking` reprezentuje proces rezerwacji i przechowuje status (np. *PENDING_ APPROVAL, PENDING_ PAYMENT, CONFIRMED*).
   - `Payment` przechowuje wynik integracji z bramką płatności (referencja dostawcy, status).

5.5 Wymiana informacji między modułami (Inter-module communication)

System realizuje komunikację między niezależnymi modułami poprzez warstwę serwisową i porty integracyjne (interfejsy), aby moduły nie były bezpośrednio ze sobą sprzężone.

- **Moduł Dopasowań ← Moduł Użytkownika (Preferencje)**
  Aby dopasować ogłoszenia do Lokatora, `MatchingService` potrzebuje preferencji Lokatora.
  Wymagana interakcja: `MatchingService` pobiera dane poprzez `UserService.getUserPreferences(userI`
- **Moduł Wynajmu → Moduł Płatności (integracja zewnętrzna)**
  Aby potwierdzić rezerwację, moduł wynajmu inicjuje płatność w zewnętrznej bramce.
  Wymagana interakcja: `PaymentService.payForBooking(bookingId, payerId)` korzysta z portu `IPaymentGateway`.
- **Moduł Wynajmu/Płatności → Moduł Powiadomień**
  Po kluczowych zdarzeniach (złożenie rezerwacji, akceptacja/odrzucenie, płatność) system powiadamia użytkowników.
  Wymagana interakcja: `BookingService` i `PaymentService` korzystają z portu `INotificationPort`.

6 Diagramy stanów i aktywności

6.1 Cel i zakres

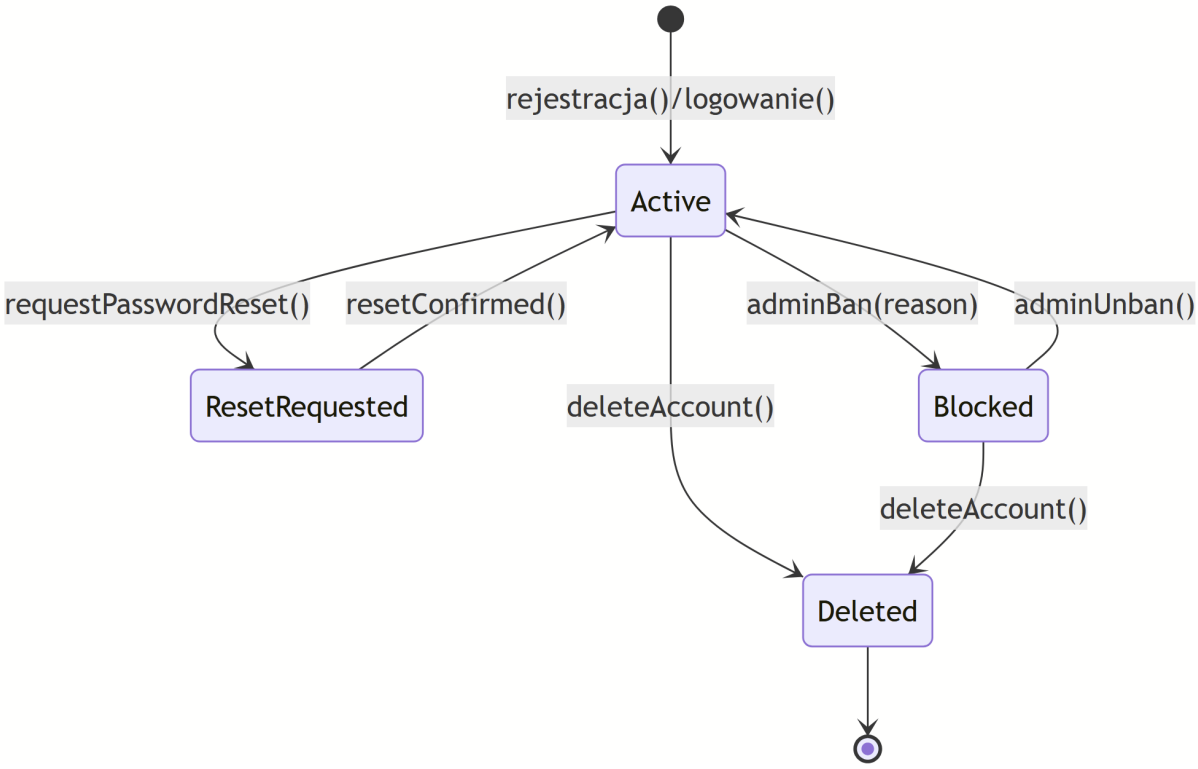
Celem jest przedstawienie:

- cyklu życia kluczowych obiektów biznesowych w formie diagramów stanów (wraz ze stanami szczególnymi oraz nietypowymi przejściami),
- kluczowych, powtarzalnych procesów biznesowych systemu w formie diagramów aktywności (z główną ścieżką i alternatywami).

W systemie FlatShare kluczowe procesy dotyczą: publikacji ogłoszeń, rezerwacji i płatności, komunikacji Lokatora z Właścicielem oraz moderacji (blokady użytkownika i obsługi zgłoszeń naruszeń).

6.2 Diagramy stanów kluczowych obiektów biznesowych

6.2.1 Obiekt: `User` (konto użytkownika)

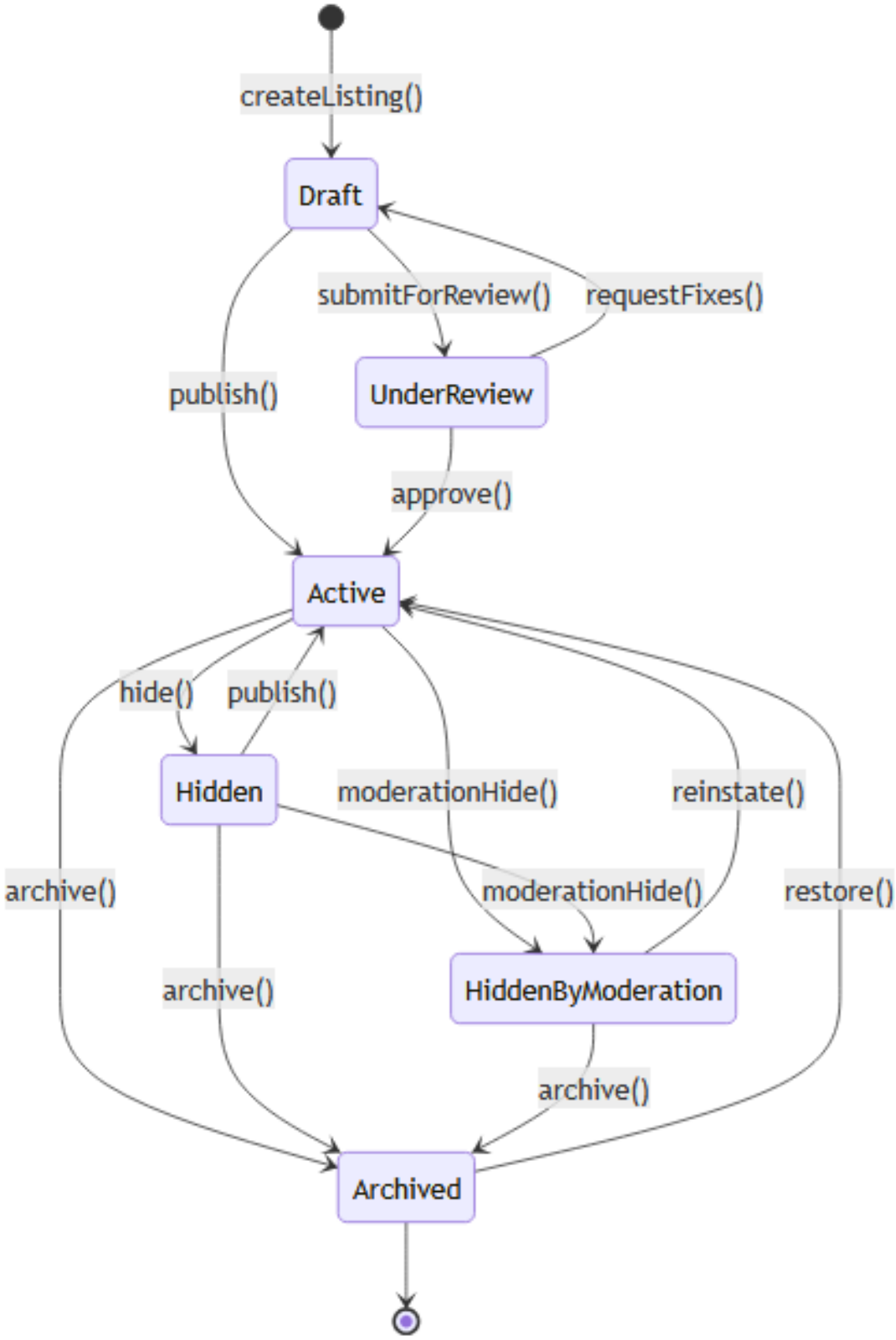


Rysunek 8: Diagram stanów: `User`.

Opis: Diagram pokazuje cykl życia konta użytkownika. Standardowo konto przechodzi do stanu `Active` po rejestracji i/lub poprawnym uwierzytelnieniu. Uwzględniono stan szczególny `Blocked`, który wynika z decyzji administratora (moderacja) i uniemożliwia korzystanie z systemu.

- **Ścieżka standardowa:** `Active` jako stan roboczy, w którym użytkownik korzysta z funkcji systemu.
- **Nietypowe przejścia:** reset hasła (`ResetRequested` → `Active`), blokada konta (`Active` → `Blocked`).
- **Stan końcowy:** `Deleted` (zamknięcie konta i zakończenie cyklu życia).

6.2.2 Obiekt: `Listing` (ogłoszenie)

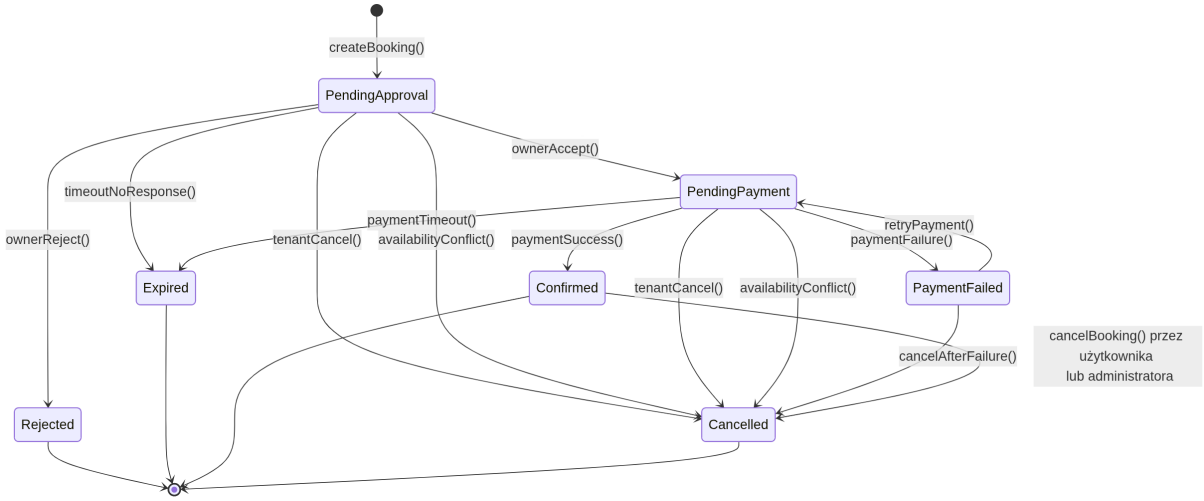


Rysunek 9: Diagram stanów: `Listing`.

Opis: Ogłoszenie jest kluczowym obiektem biznesowym w FlatShare. Stan ogłoszenia determinuje jego widoczność i możliwość rezerwacji. Diagram obejmuje zarówno etap tworzenia (roboczy), publikację, jak i zarządzanie dostępnością (`ACTIVE/HIDDEN/ARCHIVED`).

- **Ścieżka standardowa:** `Draft/UnderReview` → `Active` (publikacja) → opcjonalnie `Hidden` lub `Archived`.
- **Nietypowe przejścia:** powrót do poprawy danych po weryfikacji (`UnderReview` → `Draft`); ukrycie ogłoszenia przez system w wyniku moderacji (przejście do `Hidden`).
- **Stan szczególny:** `Hidden` używany zarówno przez Właściciela (zarządzanie dostępnością), jak i przez system (np. konsekwencja blokady konta).

6.2.3 Obiekt: `Booking` (rezerwacja)



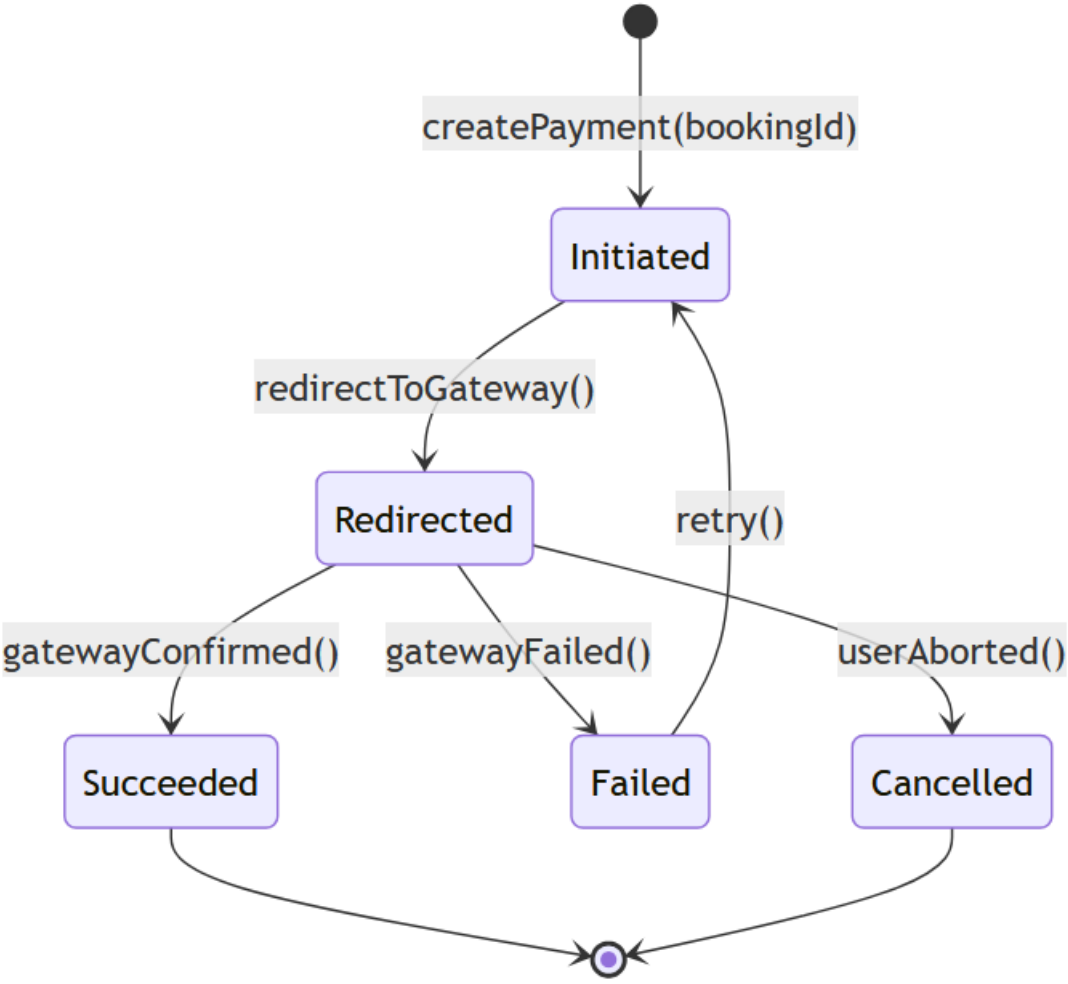
Rysunek 10: Diagram stanów: `Booking`.

Opis: Rezerwacja odzwierciedla proces biznesowy wynajmu: od złożenia prośby przez Lokatora, przez akceptację Właściciela , aż po płatność i potwierdzenie. Diagram obejmuje również zakończenia nietypowe: odrzucenie, wygaśnięcie, błąd płatności oraz wymuszone anulowanie administracyjne.

- **Ścieżka standardowa:** `PENDING_APPROVAL` → `PENDING_PAYMENT` → `CONFIRMED`.

- **Alternatywy:** `REJECTED` (odrzucenie przez Właściciela), `EXPIRED` (brak reakcji w czasie lub upływ czasu na płatność w stanie `PENDING_PAYMENT`), `PAYMENT_FAILED` (nieudana płatność z możliwością ponowienia).
- **Nietypowe zakończenie:** anulowanie procesu (np. kolizja dostępności terminu lub rezygnacja) jako stan końcowy typu `CANCELLED`.
- **Wymuszone anulowanie:** W przypadku blokady konta Właściciela przez administratora, system automatycznie wymusza przejście ze stanu `CONFIRMED` do `CANCELLED`, co uruchamia procedurę zwrotu środków.

6.2.4 Obiekt: `Payment` (płatność)



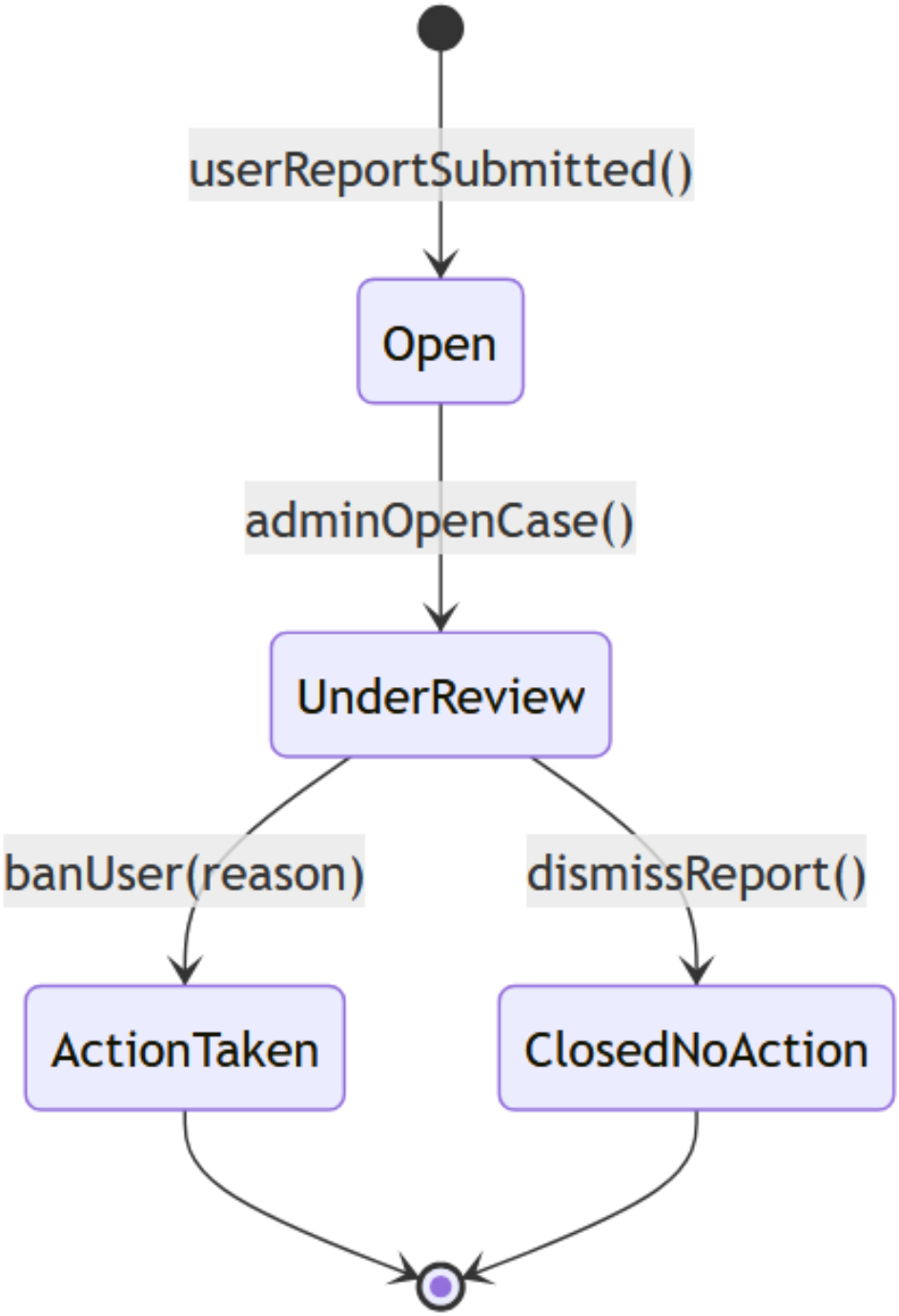
Rysunek 11: Diagram stanów: `Payment`.

Opis: Płatność jest powiązana z rezerwacją i integracją z zewnętrzną bramką płatniczą. Diagram rozróżnia etap inicjacji płatności, etap przetwarzania/redirectu oraz wyniki: sukces, porażka i anulowanie przez użytkownika.

- **Ścieżka standardowa:** `Initiated` → `Redirected` → `Succeeded`.
- **Nietypowe zakończenia:** `Failed` (błąd bramki) oraz `Cancelled` (przerwanie przez użytkownika).

- **Istotna relacja biznesowa:** `Succeeded` powoduje przejście rezerwacji do `CONFIRMED`; `Failed` mapuje się na `PAYMENT_FAILED` w obiekcie `Booking`.

6.2.5 Obiekt: `ViolationReport` (zgłoszenie naruszenia / sprawa moderacyjna)



Rysunek 12: Diagram stanów: `ViolationReport`.

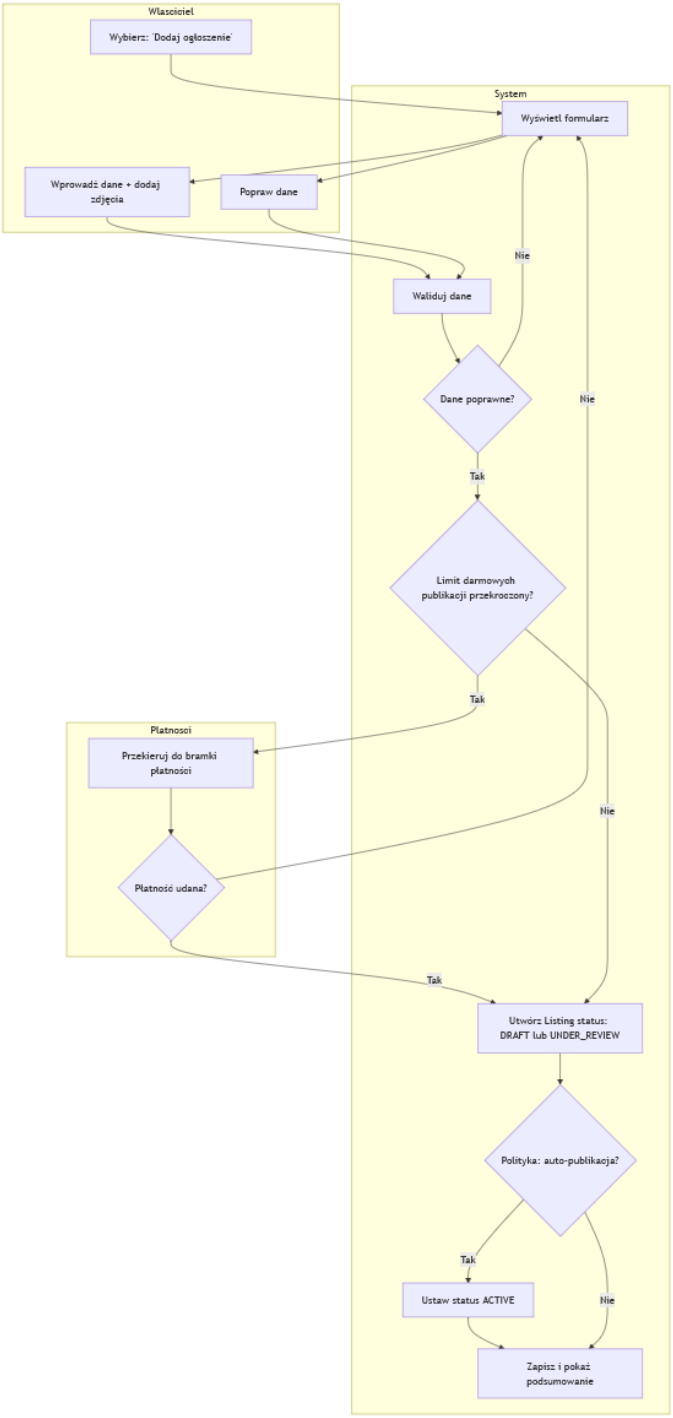
Opis: Zgłoszenie naruszenia reprezentuje przypadek biznesowy moderacji. Diagram obejmuje standardową obsługę zgłoszenia: utworzenie, analizę przez administratora i zakończenie sprawy.

- **Ścieżka standardowa:** `Open` → `UnderReview` → zakończenie.

- **Alternatywy zakończenia:** `ClosedNoAction` (zgłoszenie bezzasadne) oraz `ActionTaken` (np. blokada konta użytkownika).
- **Powiązania procesowe:** `ActionTaken` typowo skutkuje zmianą stanu `User` na `BLOCKED` oraz ukryciem ogłoszeń (`Listing` → `HIDDEN`).

6.3 Diagramy aktywności kluczowych funkcjonalności

6.3.1 Funkcjonalność: Wystawienie ogłoszenia (`Create Listing`)

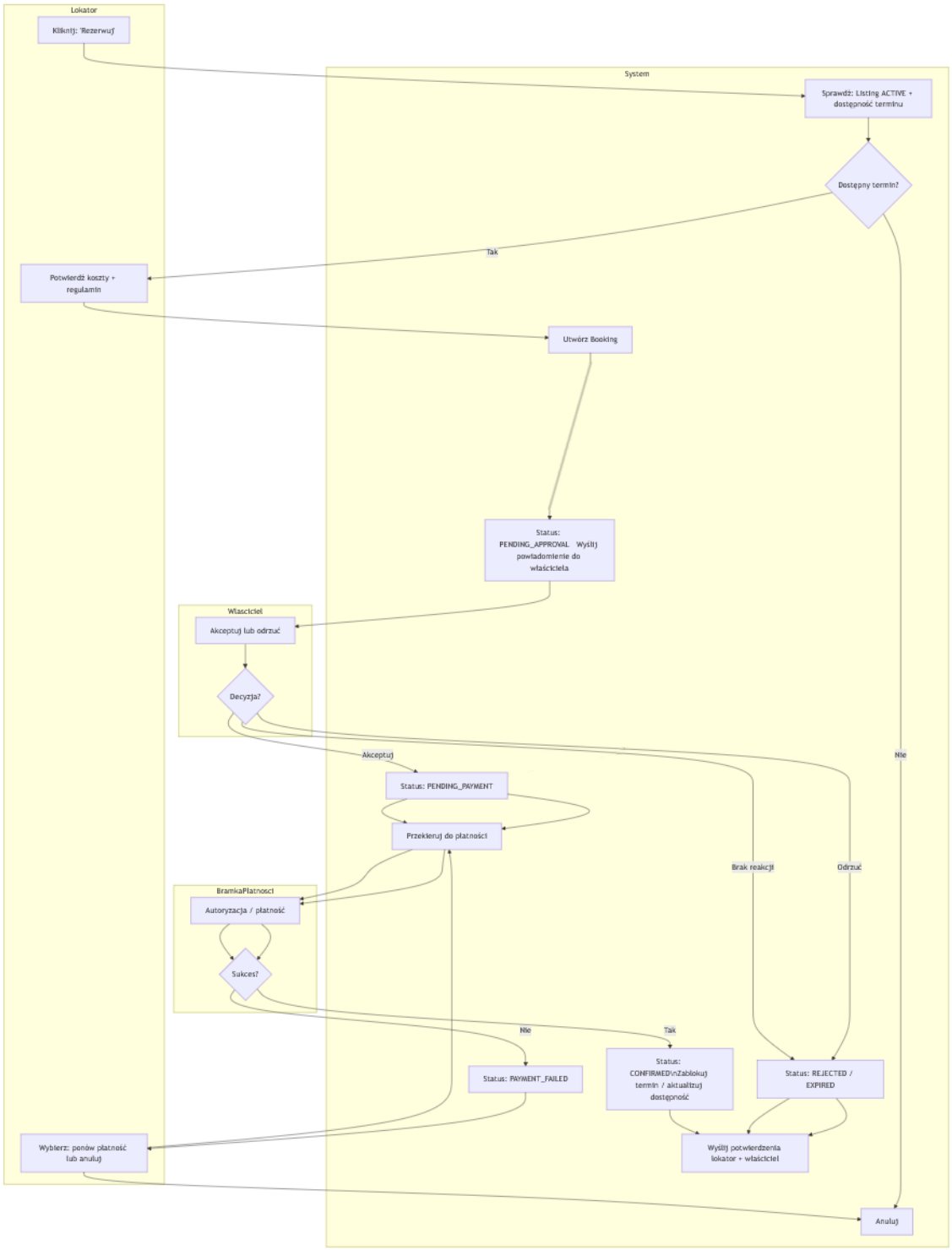


Rysunek 13: Diagram aktywności: Wystawienie ogłoszenia (`Create Listing`).

Opis: Proces wystawienia ogłoszenia jest powtarzalną funkcją Właściciela. Diagram obejmuje walidację danych, zapis ogłoszenia w stanie roboczym oraz wariant z płatnością po przekroczeniu limitu darmowych publikacji.

- **Główna ścieżka:** Właściciel wprowadza dane → system waliduje → zapisuje `Listing` jako `DRAFT` lub `UNDER_REVIEW` → opcjonalnie publikuje jako `ACTIVE`.
- **Alternatywy:** błąd walidacji (powrót do formularza), przekroczenie limitu darmowych publikacji (wymagana płatność przed publikacją).
- **Efekt biznesowy:** ogłoszenie staje się widoczne i dostępne do procesu rezerwacji dopiero w stanie `ACTIVE`.

6.3.2 Funkcjonalność: Rezerwacja pokoju z płatnością (`Book a Room`)



Rysunek 14: Diagram aktywności: Rezerwacja pokoju z płatnością (`Book a Room`).

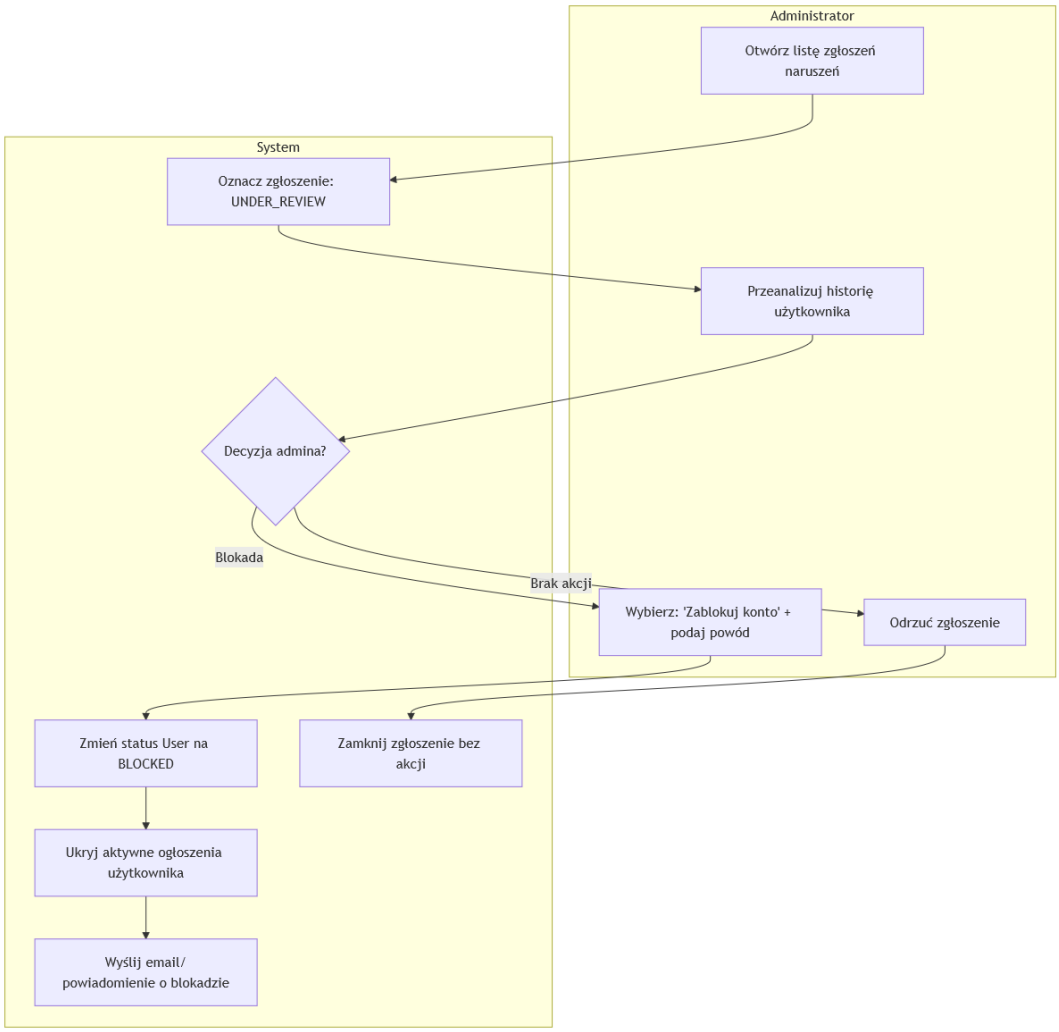
Opis: Proces rezerwacji jest kluczowym procesem biznesowym Lokatora. Diagram uwzględnia standardowy przepływ z akceptacją Właściciela, oraz alternatywy: odrzucenie, wygaśnięcie, błąd płatności i kolizję dostępności.

- **Główna ścieżka:** Lokator inicjuje rezerwację → system tworzy `Booking` jako `PENDING_APPROVAL`

→ Właściciel akceptuje → `PENDING_PAYMENT` → płatność → `CONFIRMED` → aktualizacja dostępności i powiadomienia.

- **Alternatywy:** `REJECTED` (odrzucenie), `EXPIRED` (brak reakcji), `PAYMENT_FAILED` (możliwość ponowienia), kolizja dostępności (anulowanie procesu).
- **Stan szczególny:** przejście do `CONFIRMED` jest możliwe dopiero po poprawnej płatności.

6.3.3 Funkcjonalność: Blokada użytkownika (`Ban User`)



Rysunek 15: Diagram aktywności: Blokada użytkownika (`Ban User`).

Opis: Diagram przedstawia proces moderacyjny wykonywany cyklicznie przez administratora na podstawie zgłoszeń naruszeń. Kluczową konsekwencją blokady jest ustawienie stanu konta na `BLOCKED` oraz ukrycie ogłoszeń użytkownika.

- **Główna ścieżka:** admin analizuje zgłoszenie → wybiera blokadę → system ustawia `User.status = BLOCKED` → wysyła powiadomienie i ukrywa ogłoszenia.
- **Alternatywy:** odrzucenie zgłoszenia jako bezzasadnego (zamknięcie sprawy bez blokady).
- **Efekt biznesowy:** ochrona jakości platformy (nadużycia skutkują ograniczeniem funkcji i widoczności treści).

7 Założenia wspólne

Niniejszy fragment opisuje projekt interfejsu programistycznego (API) systemu, ze szczególnym uwzględnieniem komunikacji REST, mechanizmów autoryzacji, diagramów sekwencji oraz schematów wymiany wiadomości.

Na potrzeby projektu przyjęto następujące założenia ogólne:

- API jest dostępne w wersji `/api/v1`.
- Komunikacja pomiędzy klientem a serwerem odbywa się wyłącznie z użyciem protokołu HTTPS.
- Identyfikatory zasobów są reprezentowane jako UUID w postaci tekstowej, zgodnie ze standardem RFC 4122.
- Daty są zapisywane w formacie ISO 8601 (`YYYY-MM-DD`).
- Daty wraz z czasem są zapisywane w formacie ISO 8601 w strefie UTC (`YYYY-MM-DDTHH:MM:SSZ`).
- Kwoty pieniężne są reprezentowane jako wartości liczbowe typu dziesiętnego (*decimal*) wraz z kodem waluty zgodnym ze standardem ISO 4217.

Wszystkie endpointy chronione wymagają poprawnej autoryzacji użytkownika z wykorzystaniem mechanizmu JWT. Szczegóły dotyczące procesu walidacji tokenów oraz przekazywania kontekstu użytkownika zostały opisane w osobnym rozdziale, aby uniknąć powielania tej logiki w dalszych scenariuszach biznesowych.

7.1 Konwencje zasobów REST i wersjonowanie

API projektowane jest zgodnie z podejściem *resource-oriented*:

- **Rzeczowniki w ścieżkach** (np. `/listings`, `/rentals`) reprezentują zasoby,
- **Czasowniki w metodach HTTP** (GET/POST/PATCH/DELETE) reprezentują operacje na zasobach,
- **Wersjonowanie w ścieżce** (`/api/v1`) umożliwia ewolucję interfejsu przy zachowaniu kompatybilności.

Dla list endpointów stosowana jest paginacja (`page`, `size`) oraz możliwość filtrowania parametrami zapytania.

7.2 Sytuacje wyścigu

Dodatkowo, w procesach zależnych od dostępności zasobu (np. rezerwacja terminu) system musi obsłużyć sytuacje wyścigu (`409 Conflict`) — przykładowo, gdy dwie osoby próbują zarezerwować ten sam pokój w tym samym okresie.

7.3 Wspólny format odpowiedzi błędów

Odpowiedzi błędów są ustandaryzowane, aby klient mógł automatycznie interpretować problem. Minimalny zestaw pól obejmuje: `timestamp`, `status`, `error` oraz (opcjonalnie) `fieldErrors` dla walidacji. W praktyce implementacja może również zwracać `message` oraz `path` (ścieżkę żądania), jednak w przykładach utrzymano format uproszczony.

7.4 Słownik typów danych wykorzystywanych w komunikatach

W tabeli poniżej zestawiono podstawowe typy danych wykorzystywane w komunikacji pomiędzy klientem a serwerem.

Typ	Reprezentacja	Uwagi
UUID	`"550e8400-e29b-41d4-a716-446655440000"`	Identyfikatory zasobów (RFC 4122).
Date	`"2024-11-01"`	Data w ISO 8601 (`YYYY-MM-DD`).
DateTime (UTC)	`"2024-10-15T14:00:00Z"`	Znacznik czasu w ISO 8601 w UTC.
Money	`amount + currency`	Kwota dziesiętna + kod waluty ISO 4217 (np. PLN).
Enum statusu	`"ACTIVE"`	Stany obiektów domenowych (np. `Listing.status`, `Booking.status`).
Stronicowanie	`page/size/total`	Metadane paginacji w odpowiedziach list.

W kolejnych rozdziałach dokumentu przedstawiono zarówno przykładowe żądania i odpowiedzi HTTP, jak i diagramy sekwencji ilustrujące przebieg logiki biznesowej po stronie systemu.

8 Wspólny mechanizm autoryzacji (JWT)

W systemie zastosowano mechanizm autoryzacji oparty o tokeny JWT (*JSON Web Token*), który stanowi wspólny element dla wszystkich endpointów wymagających uwierzytelnienia użytkownika. Mechanizm ten został wydzielony do osobnego scenariusza w celu uniknięcia powielania tej samej logiki w dalszych diagramach sekwencji oraz opisach przypadków użycia.

8.1 Zakres stosowania

Autoryzacja z wykorzystaniem JWT dotyczy wszystkich endpointów chronionych interfejsu API. Każde żądanie do takiego endpointu musi zawierać poprawny token JWT przekazany w nagłówku HTTP `Authorization`.

8.2 Wymagane nagłówki HTTP

Dla endpointów chronionych wymagane są następujące nagłówki:

- `Authorization: Bearer <JWT>`
- `Accept: application/json`

Dodatkowo, opcjonalnie może zostać przekazany nagłówek:

- `X-Request-Id: <UUID>` – identyfikator żądania wykorzystywany w celach diagnostycznych oraz do śledzenia przepływu żądań w systemie.

8.3 Proces walidacji tokenu

Proces weryfikacji tokenu JWT realizowany jest na poziomie filtra autoryzacyjnego, jeszcze przed przekazaniem żądania do warstwy kontrolerów. Logika biznesowa poszczególnych endpointów zakłada, że kontekst użytkownika został już poprawnie ustawiony.

W ramach walidacji tokenu wykonywane są następujące kroki:

1. Przechwycenie przychodzącego żądania HTTP.

2. Odczyt nagłówka `Authorization` oraz wyodrębnienie tokenu JWT.
3. Weryfikacja poprawności podpisu kryptograficznego tokenu.
4. Sprawdzenie daty ważności tokenu.
5. Odczyt identyfikatora użytkownika oraz ról zapisanych w tokenie.
6. Ustawienie kontekstu bezpieczeństwa aplikacji (np. `SecurityContext`).

8.4 Obsługa błędów autoryzacji

W przypadku braku tokenu JWT, przekazania tokenu niepoprawnego lub tokenu, którego ważność wygasła, żądanie zostaje odrzucone, a system zwraca odpowiedź:

- `401 Unauthorized`

Żądanie nie jest wówczas przekazywane do dalszych warstw aplikacji.

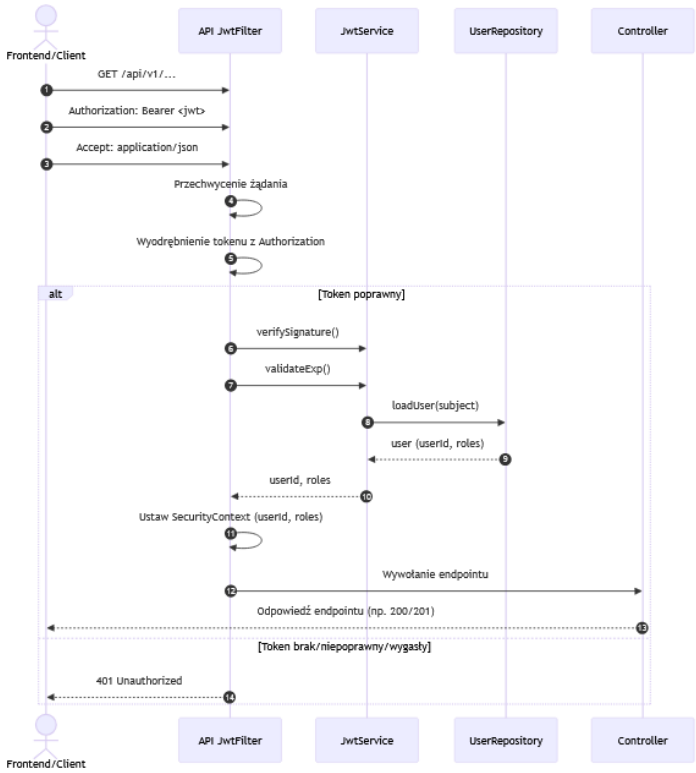
8.5 Uwagi projektowe

Wydzielenie wspólnego mechanizmu autoryzacji pozwala na:

- zwiększenie czytelności dokumentacji,
- uproszczenie diagramów sekwencji opisujących logikę biznesową,
- centralne zarządzanie polityką bezpieczeństwa systemu.

8.6 Diagram sekwencji – walidacja tokenu JWT

Na rysunku 16 przedstawiono wspólny mechanizm walidacji tokenu JWT, realizowany przed wykonaniem logiki biznesowej endpointów chronionych.



Rysunek 16: Diagram sekwencji walidacji tokenu JWT

9 Rejestracja

Rejestracja jest podstawową funkcjonalnością, bez której użytkownik nie może wejść w dalszą interakcję z systemem. Jest potrzebna zarówno lokatorom, jak i właścicielom pokoi.

9.1 Opis endpointu

Rejestracja realizowana jest za pomocą restful endpointu powiązanego z użytkownikami.

- Metoda HTTP: `POST`
- Adres URL: `/api/v1/users`
- Typ treści: `application/json`

9.2 Przykładowe żądanie

Poniżej przedstawiono przykładowe żądanie HTTP służące do utworzenia nowego użytkownika

Listing 1: Przykładowe żądanie rejestracji

```
POST /api/v1/users
Content-Type: application/json
Accept: application/json
```

Listing 2: Ciało żądania rejestracji

```
{
  "firstName": "Jan",
  "lastName": "Kowalski",
```

```
  "email": "jan.kowalski@example.com",
  "password": "jestemSobieWesołyJanek",
  "role": "LANDLORD"
}
```

Pole *role* przyjmuje następujące wartości:

- `"LANDLORD"` - Właściciel
- `"TENANT"` - Lokator

9.3 Przykładowa odpowiedź — sukces

W przypadku poprawnego zarejestrowania użytkownika system zwraca informację o sukcesie.

Listing 3: Odpowiedź HTTP — sukces

```
201 Created
Content-Type: application/json
Location: http://flatshareapp.example.com/users/123

{
  "message": "New user created",
  "user": {
    "id": 123,
    "firstName": "Jan",
    "lastName": "Kowalski",
    "email": "jan.kowalski@example.com",
    "role": "LANDLORD"
  }
}
```

9.4 Obsługa błędów

W przypadku nieudanej rejestracji żądanie zostaje odrzucone, a system zwraca odpowiedź:

Listing 4: Odpowiedź HTTP — nieudana rejestracja

```
400 Bad Request
```

10 Pobieranie danych użytkownika

10.1 Opis endpointu

Służy do pobierania podstawowych informacji o użytkowniku - odzwierciedla zawartość nagłówka *Location* w odpowiedzi endpointu rejestracji. W url nalezy wstawić id użytkownika.

- Metoda HTTP: `GET`
- Adres URL: `/api/v1/users/1234-1234-1234-1234`
- Typ treści: `application/json`

10.2 Przykładowe żądanie

Listing 5: Przykładowe żądanie rejestracji

```
POST /api/v1/users/1234-1234-1234-1234
Content-Type: application/json
Accept: application/json
```

10.3 Przykładowa odpowiedź — sukces

W przypadku poprawnego id, zwraca informacje o użytkowniku.

Listing 6: Odpowiedź HTTP — sukces

```
200 OK
Content-Type: application/json

{
  "id": 123,
  "firstName": "Jan",
  "lastName": "Kowalski",
  "email": "jan.kowalski@example.com",
  "role": "LANDLORD"
}
```

Pole *role* przyjmuje następujące wartości:

- `"LANDLORD"` - Właściciel
- `"TENANT"` - Lokator

10.4 Obsługa błędów

W przypadku złego id, zwracany jest NotFound.

Listing 7: Odpowiedź HTTP — nieudana rejestracja

```
404 Not Found
```

11 Logowanie

Proces logowania umożliwia uwierzytelnienie użytkownika w systemie oraz uzyskanie tokenu JWT (*access token*), który jest następnie wykorzystywany do autoryzacji żądań kierowanych do endpointów chronionych interfejsu API. Aby logowanie było restful i polegało wyłącznie na tworzeniu/dostępie do zasobów, reprezentujemy logowanie jako endpoint tworzący nową sesję.

11.1 Opis endpointu

Logowanie realizowane jest za pomocą dedykowanego endpointu autoryzacyjnego:

- Metoda HTTP: `POST`
- Adres URL: `/api/v1/sessions`
- Typ treści: `application/json`

11.2 Przykładowe żądanie

Poniżej przedstawiono przykładowe żądanie HTTP służące do uwierzytelnienia użytkownika w systemie (utworzenia sesji)

Listing 8: Przykładowe żądanie logowania

```
POST /api/v1/sessions
Content-Type: application/json
Accept: application/json
```

Listing 9: Ciało żądania logowania

```
{
  "email": "jan.kowalski@example.com",
  "password": "P@ssw0rd!"
}
```

11.3 Przykładowa odpowiedź — sukces

W przypadku poprawnego uwierzytelnienia użytkownika system zwraca informację o utworzeniu sesji reprezentowanej przez token JWT, który powinien zostać zapisany po stronie klienta i dołączany do kolejnych żądań.

Listing 10: Odpowiedź HTTP — sukces

```
201 Created
Content-Type: application/json
Location: http://flatshareapp.example.com/sessions/0e1ebd25-19bd-40b6-8a9b-
    c5831f3e0b5d
```

Listing 11: Ciało odpowiedzi — sukces

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "sessionId": 0e1ebd25-19bd-40b6-8a9b-c5831f3e0b5d
  "type": "Bearer",
  "expiresIn": 3600,
  "roles": ["LANDLORD"]
}
```

11.4 Obsługa błędów

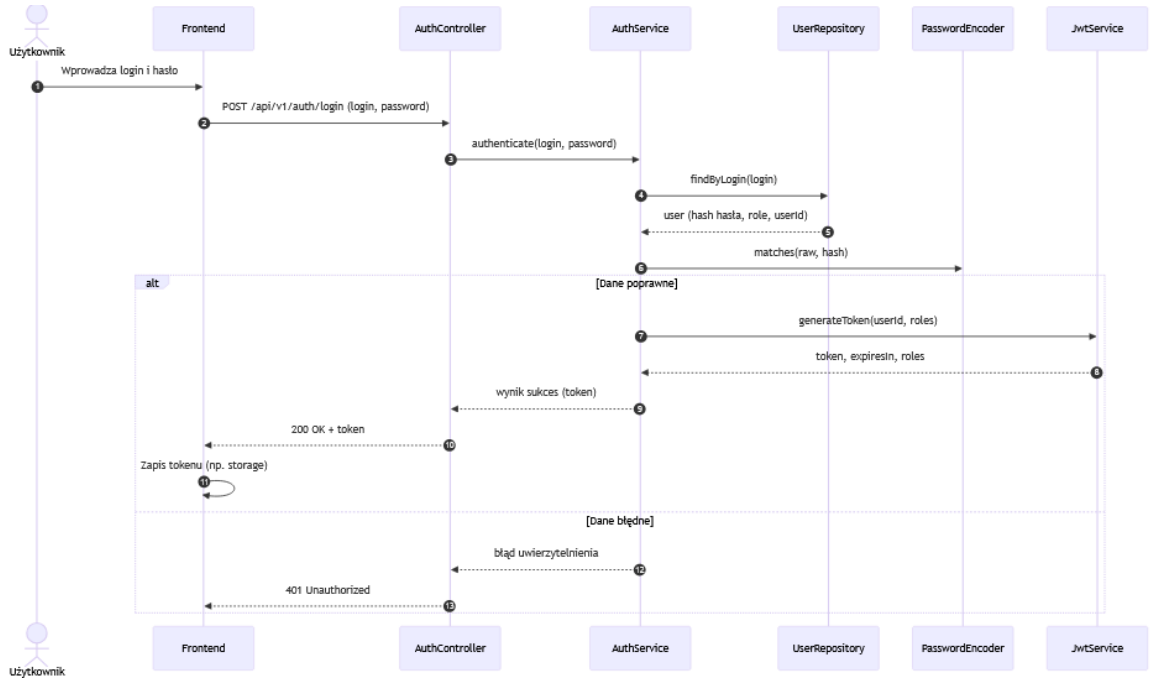
W przypadku podania nieprawidłowych danych uwierzytelniających (błędny login lub hasło) żądanie zostaje odrzucone, a system zwraca odpowiedź:

Listing 12: Odpowiedź HTTP — błędne dane logowania

```
401 Unauthorized
```

11.5 Diagram sekwencji procesu logowania

Na rysunku 17 przedstawiono diagram sekwencji obrazujący proces logowania użytkownika, obejmujący weryfikację danych uwierzytelniających oraz generowanie tokenu JWT.



Rysunek 17: Diagram sekwencji procesu logowania

11.6 Uwagi projektowe

Mechanizm logowania został zaprojektowany w sposób umożliwiający jednoznaczne oddzielenie procesu uwierzytelnienia od logiki biznesowej systemu. Token JWT zwracany w odpowiedzi zawiera informacje niezbędne do identyfikacji użytkownika oraz jego ról, co pozwala na efektywne zarządzanie dostępem do zasobów systemu. Jednakże, w praktyce potrzebny jest jeszcze sposób na odświeżenie sesji (tokenu) użytkownika, dlatego prezentujemy następujący endpoint:

12 Odświeżanie sesji

12.1 Opis endpointu

- Metoda HTTP: `PATCH`
- Adres URL: `/api/v1/sessions`
- Typ treści: `application/json`
- Endpoint chroniony: tak (JWT)

12.2 Przykładowe żądanie

Poniżej przedstawiono przykładowe żądanie HTTP służące do uwierzytelnienia użytkownika w systemie (utworzenia sesji)

Listing 13: Przykładowe żądanie logowania

```
PATCH /api/v1/sessions/4bdb3f80-9ea8-49bb-91d5-591fff9e8815
```

12.3 Przykładowa odpowiedź — sukces

W przypadku poprawnego uwierzytelnienia użytkownika system zwraca informację o odświeżeniu sesji reprezentowanej przez token JWT, który powinien zostać zapisany po stronie klienta i dołączany do kolejnych żądań.

Listing 14: Odpowiedź HTTP — sukces

```
201 Created
Content-Type: application/json
```

Listing 15: Ciało odpowiedzi — sukces

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "sessionId": 4bdb3f80-9ea8-49bb-91d5-591fff9e8815
  "type": "Bearer",
  "expiresIn": 3600,
  "roles": ["LANDLORD"]
}
```

12.4 Obsługa błędów

W przypadku podania nieprawidłowych danych uwierzytelniających (błędny token) żądanie zostaje odrzucone, a system zwraca odpowiedź:

Listing 16: Odpowiedź HTTP — błędna autoryzacja

```
401 Unauthorized
```

13 Pobieranie informacji o sesji

13.1 Opis endpointu

Endpoint ten jest dla formalizmu, aby nagłówek *Location* przy tworzeniu sesji (Logowanie) miał sens.

- Metoda HTTP: `GET`
- Adres URL: `/api/v1/sessions`
- Typ treści: `application/json`
- Endpoint chroniony: tak (JWT)

13.2 Przykładowe żądanie

Listing 17: Przykładowe żądanie logowania

```
GET /api/v1/sessions/4bdb3f80-9ea8-49bb-91d5-591fff9e8815
```

13.3 Przykładowa odpowiedź — sukces

Listing 18: Odpowiedź HTTP — sukces

```
200 OK
Content-Type: application/json
```

Listing 19: Ciało odpowiedzi — sukces

```
{
  "sessionId": 4bdb3f80-9ea8-49bb-91d5-591fff9e8815,
  "userId": 425523-3424-2137-91d5-213769696
}
```

13.4 Obsługa błędów

W przypadku podania nieprawidłowych danych (użytkownik musi być właścicielem sesji aby móc wywołać tę metodę) uwierzytelniających (błędny token) żądanie zostaje odrzucone, a system zwraca odpowiedź:

Listing 20: Odpowiedź HTTP — błędna autoryzacja

```
401 Unauthorized
```

13.5 Reset hasła

Reset hasła realizowany jest jako proces dwuetapowy, aby uniknąć ujawniania informacji o istnieniu konta oraz zapewnić kontrolę nad operacją zmiany hasła. Wariant ten odpowiada historii US-Sys2.

13.5.1 Etap 1: żądanie resetu

- Metoda HTTP: `POST`
- Adres URL: `/api/v1/auth/password-reset/request`
- Typ treści: `application/json`

Listing 21: Ciało żądania — żądanie resetu hasła

```
{
  "email": "jan.kowalski@example.com"
}
```

Odpowiedź w tym etapie ma charakter „neutralny” (np. `202 Accepted`) — system nie potwierdza, czy podany adres istnieje w bazie. Jeśli konto istnieje, użytkownik otrzymuje wiadomość (email) z tokenem resetu.

13.5.2 Etap 2: potwierdzenie resetu

- Metoda HTTP: `POST`
- Adres URL: `/api/v1/auth/password-reset/confirm`
- Typ treści: `application/json`

Listing 22: Ciało żądania — potwierdzenie resetu hasła

```
{
  "resetToken": "b7c9... (token jednorazowy)",
  "newPassword": "N3wP@ssw0rd!"
}
```

Reguły bezpieczeństwa:

- token resetu jest jednorazowy i ograniczony czasowo,
- po skutecznej zmianie hasła wszystkie aktywne sesje/tokenu (poza bieżącą) mogą zostać unieważnione zgodnie z polityką bezpieczeństwa,
- system stosuje politykę złożoności hasła i nie przechowuje haseł w postaci jawnej (hash + sól).

14 Zarządzanie preferencjami

Niniejszy rozdział opisuje proces dopasowywania ofert do preferencji użytkownika pełniącego rolę lokatora. Opis dotyczy logiki biznesowej realizowanej po stronie systemu. Mechanizm technicznej autoryzacji JWT został opisany w osobnym rozdziale i nie jest tutaj powielany.

Zakłada się, że żądanie zostało przetworzone przez warstwę autoryzacji, a kontekst użytkownika (identyfikator użytkownika) jest dostępny w warstwie kontrolera.

14.1 Opis endpointu - ustawianie preferencji

Aby dopasowywanie było deterministyczne i możliwe do odtworzenia, preferencje są utrzymywane po stronie serwera. Preferencje są danymi powiązanymi z rolą `TenantRole` (a nie „globalnym” profilem konta), co pozwala na rozszerzanie modelu bez wpływu na pozostałe role.

- Metoda HTTP: `PUT`
- Adres URL: `/api/v1/users/me/preferences`
- Endpoint chroniony: tak (JWT)

Listing 23: Ciało żądania — preferencje Lokatora

```
{
  "maxPrice": 1500.00,
  "currency": "PLN",
  "smokingAllowed": false,
  "petsAllowed": true,
  "preferredDistricts": ["Mokotów", "Ochota"]
}
```

14.2 Opis endpointu - pobieranie preferencji

- Metoda HTTP: `GET`
- Adres URL: `/api/v1/users/me/preferences`
- Endpoint chroniony: tak (JWT)

Listing 24: Ciało odpowiedzi — sukces

```
{
  "maxPrice": 1500.00,
  "currency": "PLN",
  "smokingAllowed": false,
  "petsAllowed": true,
  "preferredDistricts": ["Mokotów", "Ochota"]
}
```

14.3 Uwagi projektowe

Uwagi: preferencje mogą być niekompletne (np. brak listy dzielnic), a w takim przypadku algorytm dopasowań traktuje brakujące pola jako neutralne. Dla zapewnienia spójności, po aktualizacji preferencji kolejne wywołanie endpointu dopasowań powinno uwzględniać nowy zestaw danych.

15 Wyszukiwanie pasujących mieszkań

15.1 Opis endpointu

- Metoda HTTP: `GET`
- Adres URL: `/api/v1/matches`
- Endpoint chroniony: tak (JWT)

15.2 Parametry zapytania

- `page` – numer strony wyników (numerowanie od 0),
- `size` – liczba elementów na stronie.
- `city` – miasto.
- `district` – dzielnica miasta.
- `minPrice` – minimalna cena.
- `maxPrice` – maksymalna cena.
- `petsAllowed` – zezwolenie na mieszkanie ze zwierzętami.
- `nonSmokingOnly` – wymaganie aby wszyscy lokatorzy byli niepalący.
- `closeToShops` – liczba elementów na stronie.
- `profile` – jedna z wartości: "student", ... (otwarte na rozszerzenie)
- `minArea` – minimalna powierzchnia pokoju $[m^2]$.
- `maxArea` – maksymalna powierzchnia pokoju $[m^2]$.
- `startDate` - początek okresu wynajmu

15.3 Przykładowe żądanie

Listing 25: Żądanie HTTP — pobranie dopasowań

```
GET /api/v1/matches?page=0&size=10?city=Warsaw&district=Mokotow&minPrice=1000&maxPrice
    =2500&petsAllowed=true&nonSmokingOnly=true&closeToShops=true&profile=student&
    minArea=10&maxArea=25&startDate=2024-11-01
Authorization: Bearer <JWT>
Accept: application/json
```

15.4 Przykładowa odpowiedź — sukces

W odpowiedzi system zwraca listę dopasowanych ogłoszeń wraz z metadanymi stronicowania.

Listing 26: Odpowiedź HTTP — sukces

```
200 OK
Content-Type: application/json
```

Listing 27: Ciało odpowiedzi — lista dopasowań

```
{
  "content": [
    {
      "listingId": "c92f1c80-2d3a-4e5b-9a1f-8b2c3d4e5f6a",
      "title": "Cichy pokój",
      "price": 420.00,
      "currency": "PLN",
      "area": 12.36,
      "availableUntil": "2027-09-21",
      "availableSince": "2026-04-28",
      "matchScore": 0.95,
      "location": {
        "city": "Warszawa",
        "district": "Mokotów",
        "street": "Puławska 21"
        "aptNumber": "37"
      },
      attributes: {
          "petsAllowed": true,
          "nonSmokingOnly": true,
          "closeToShops": false,
          "profile": "student",
      }
    }
  ],
  "page": {
    "size": 10,
    "number": 0,
    "totalElements": 45,
    "totalPages": 5
  }
}
```

15.5 Obsługa sytuacji brzegowych

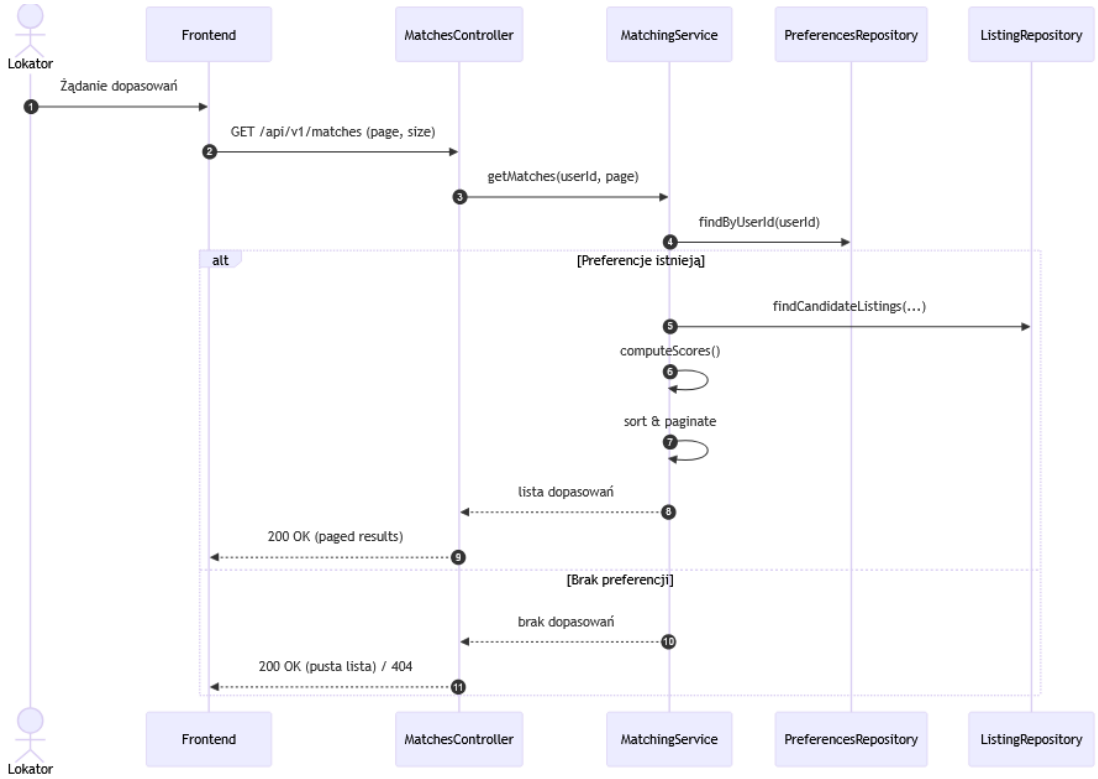
Jeżeli użytkownik nie posiada zapisanych preferencji, system może zwrócić:

- pustą listę dopasowań (kod `200 OK`), lub
- odpowiedź `404 Not Found`.

Wybór zachowania zależy od przyjętej polityki biznesowej systemu.

15.6 Diagram sekwencji dopasowywania

Na rysunku 18 przedstawiono diagram sekwencji ilustrujący proces pobierania dopasowanych ogłoszeń po stronie systemu.



Rysunek 18: Diagram sekwencji procesu dopasowywania

15.7 Uwagi projektowe

Proces dopasowywania opiera się na analizie preferencji użytkownika oraz ocenie stopnia zgodności ofert z wykorzystaniem algorytmu punktowego (*match score*). Wyniki są sortowane malejąco według stopnia dopasowania i zwracane w postaci stronicowanej.

16 Wystawienie ogłoszenia i zarządzanie nim

Niniejszy rozdział opisuje proces wystawienia ogłoszenia przez użytkownika pełniącego rolę wynajmującego. Opis dotyczy wyłącznie logiki biznesowej realizowanej po stronie systemu. Mechanizm technicznej autoryzacji JWT został opisany w osobnym rozdziale i nie jest tutaj powielany.

Zakłada się, że żądanie dotarło do warstwy kontrolera z poprawnie ustawionym kontekstem użytkownika (identyfikator użytkownika oraz role).

16.1 Opis endpointu

- Metoda HTTP: `POST`
- Adres URL: `/api/v1/listings`
- Endpoint chroniony: tak (JWT)

16.2 Przykładowe żądanie

Listing 28: Żądanie HTTP — wystawienie szczegółów ogłoszenia

```
POST /api/v1/listings
Authorization: Bearer <JWT>
Content-Type: application/json
Accept: application/json
```

Listing 29: Ciało żądania — wystawienie ogłoszenia

```
{
  "title": "Pokój przy metrze",
  "description": "Najlepszy pokój, nawet karaluchów nie ma",
  "price": 1200.00,
  "currency": "PLN",
  "availableSince": "2024-10-01"
  "availableUntil": "2057-12-28",
  "ownerContact": "numer telefonu: 424 242 424",
  "area": 12.36,
  "location": {
    "city": "Warszawa",
    "district": "Mokotów",
    "street": "Puławska 21"
    "aptNumber": "37"
  },
  attributes: {
      "petsAllowed": true,
      "nonSmokingOnly": true,
      "closeToShops": false,
      "profile": "student",
  }
}
```

16.3 Przykładowa odpowiedź — sukces

W przypadku poprawnej walidacji danych oraz spełnienia warunków biznesowych ogłoszenie zostaje zapisane w systemie.

Listing 30: Odpowiedź HTTP — sukces

```
201 Created
Location: /api/v1/listings/c92f1c80-2d3a-4e5b-9a1f-8b2c3d4e5f6a
Content-Type: application/json
```

Listing 31: Ciało odpowiedzi — sukces

```
{
  "listingId": "c92f1c80-2d3a-4e5b-9a1f-8b2c3d4e5f6a",
  "status": "ACTIVE",
  "createdAt": "2024-09-15T10:30:00Z"
}
```

Listing 32: Żądanie HTTP — pobranie szczegółów ogłoszenia

```
GET /api/v1/listings/{listingId}
Authorization: Bearer <JWT>
Content-Type: application/json
Accept: application/json
```

Listing 33: Ciało żądania — pobranie szczegółów ogłoszenia

```
{
    "listingId": 33db8939-5830-47b6-8e1b-facb6ac94699,
    "title": "Pokój przy metrze",
    "description": "Najlepszy pokój, nawet karaluchów nie ma",
    "price": 1200.00,
    "currency": "PLN",
    "availableSince": "2024-10-01"
    "availableUntil": "2057-12-28",
    "ownerContact": "numer telefonu: 424 242 424",
    "area": 12.36,
    "location": {
    "city": "Warszawa",
    "district": "Mokotów",
    "street": "Puławska 21"
    "aptNumber": "37"
    },
    attributes: {
        "petsAllowed": true,
        "nonSmokingOnly": true,
        "closeToShops": false,
        "profile": "student",
    }
}
```

Listing 34: Odpowiedź HTTP — sukces

```
200 OK
```

16.4 Obsługa błędów

W trakcie przetwarzania żądania mogą wystąpić następujące błędy:

- `400 Bad Request` – niepoprawne dane wejściowe (błędy walidacji),
- `403 Forbidden` – brak uprawnień do wskazanego zasobu,
- `404 Not Found` – wskazane mieszkanie nie istnieje.

Listing 35: Przykładowa odpowiedź — błąd walidacji

```
{
  "timestamp": "2024-09-15T10:30:00Z",
  "status": 400,
  "error": "ValidationError",
  "fieldErrors": [
    { "field": "price", "message": "Cena musi być większa od 0" },
    { "field": "title", "message": "Tytuł jest wymagany" }
  ]
}
```

Pozostałe operacje na zasobie `Listing`. W praktycznym użyciu systemu niezbędne są również endpointy do pobierania i zarządzania widocznością ogłoszeń (US-L1, US-W3). Poniżej zebrano kluczowe ścieżki (bez rozwijania diagramów sekwencji):

- `GET /api/v1/listings?city=Warszawa&district=Praga&street=Puławska&aptNumber=42`

Listing 36: Odpowiedź żądania

```
[
    {
      "listingId": 33db8939-5830-47b6-8e1b-facb6ac94699,
      "title": "Pokój przy metrze",
      "description": "Najlepszy pokój, nawet karaluchów nie ma",
      "price": 1200.00,33db8939-5830-47b6-8e1b-facb6ac94699
      "currency": "PLN",
      "availableSince": "2024-10-01"
      "availableUntil": "2057-12-28",
      "ownerContact": "numer telefonu: 424 242 424",
      "area": 12.36,
      "location": {
        "city": "Warszawa",
        "district": "Mokotów",
        "street": "Puławska 21"
        "aptNumber": "42"
      },
      attributes: {
          "petsAllowed": true,
          "nonSmokingOnly": true,
          "closeToShops": false,
          "profile": "student",
      }
    }
]
```

  Możliwe jest filtrowanie po następujących parametrach:

  - city
  - district
  - street
  - aptNumber
  - ownerID

- `PATCH /api/v1/listings/{listingId}` – edycja pól ogłoszenia (tylko właściciel zasobu). W tym żądaniu podaje się tylko te pola które chcemy poddać edycji. Pola Id nie można zedytować. Nie ustalono kiedy można edytować ogłoszenie. Odpowiedź jest taka sama jak w prypadku zwykłego pobrania szczegółów.

Listing 37: Żądanie edycji ogłoszenia

```
    {
      "price": 10000000.0,
      "availableSince": "3050-03-20"
    }
```

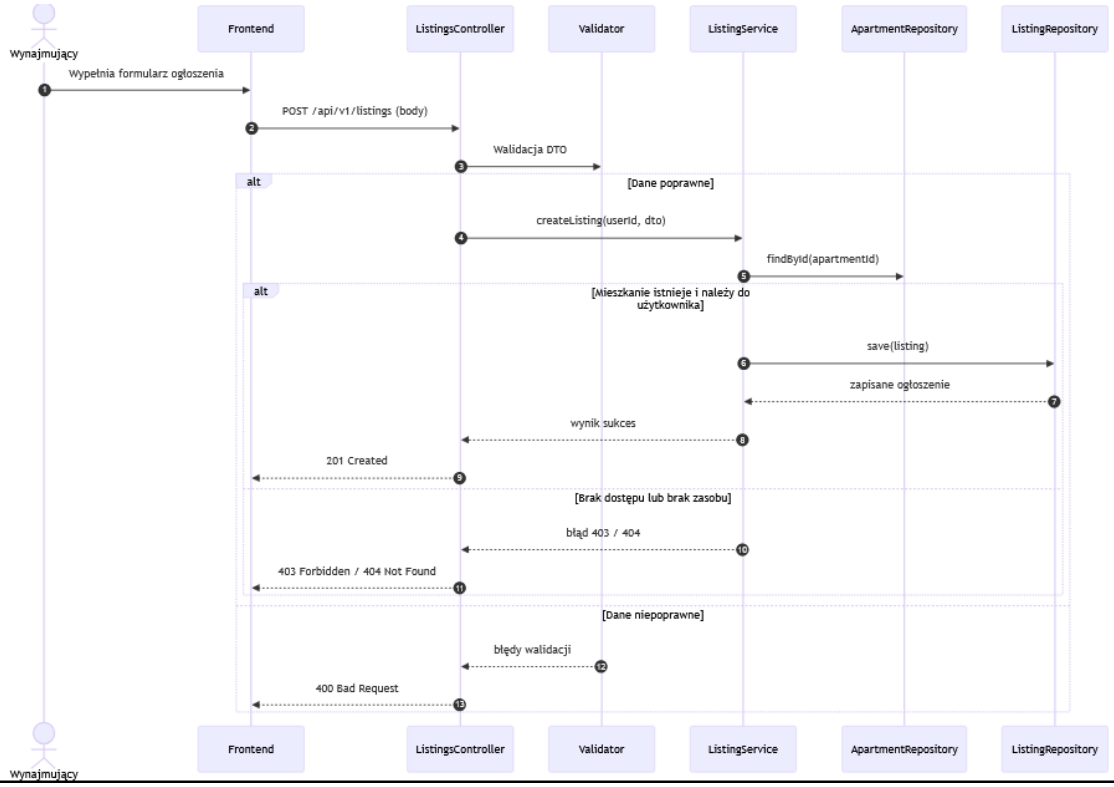
- `GET /api/v1/listings/{listingId}` – pobranie listingu o podanym id
- `PATCH /api/v1/listings/{listingId}/submit` – podanie ogłoszenia pod recenzję administratora

- `PATCH /api/v1/listings/{listingId}/request-fixes` – żądanie poprawek przez administratora
- `PATCH /api/v1/listings/{listingId}/approve` – akceptacja ogłoszenia przez administratora
- `PATCH /api/v1/listings/{listingId}/hide` – ukrycie ogłoszenia
- `PATCH /api/v1/listings/{listingId}/publish` – opublikowanie ukrytego ogłoszenia
- `PATCH /api/v1/listings/{listingId}/archive` – archiwizacja ogłoszenia
- `GET /api/v1/listings/all` – pobranie wszystkich ogłoszeń

Te endpointy pozwalają zaimplementować cykl ogłoszenia ogłoszenia. Tożsamość (jwt bearer) użytkownika wywołującego metodę pozwala określić czy jest to administrator czy właściciel ogłoszenia.

16.5 Diagram sekwencji wystawienia ogłoszenia

Na rysunku 19 przedstawiono diagram sekwencji ilustrujący przebieg procesu wystawienia ogłoszenia po stronie systemu.



Rysunek 19: Diagram sekwencji procesu wystawienia ogłoszenia

16.6 Uwagi projektowe

Walidacja danych wejściowych realizowana jest przed wykonaniem logiki biznesowej, co pozwala na szybkie odrzucenie niepoprawnych żądań.

17 Zarządzanie zdjęciami ogłoszeń

Listing 38: Żądanie HTTP - Dodanie Zdjęcia do ogłoszenia

```
POST /api/v1/listings/33db8939-5830-47b6-8e1b-facb6ac94699/photos
Authorization: Bearer <JWT>
Content-Type: multipart/form-data
Accept: application/json
```

Listing 39: Odpowiedź HTTP — sukces

```
201 Created
Location: /api/v1/listings/33db8939-5830-47b6-8e1b-facb6ac94699/photos/98606e9e-e43d
    -440c-8906-ee6dfc110198
```

Listing 40: Żądanie HTTP - Pobranie danego zdjęcia

```
GET /api/v1/listings/33db8939-5830-47b6-8e1b-facb6ac94699/photos/98606e9e-e43d-440c
    -8906-ee6dfc110198
Authorization: Bearer <JWT>
Content-Type: application/json
Accept: application/json
```

Listing 41: Odpowiedź HTTP — sukces

```
Content-Type: image/jpeg
200 OK
```

Listing 42: Żądanie HTTP - Wypisanie listy zdjęć danego ogłoszenia

```
GET /api/v1/listings/33db8939-5830-47b6-8e1b-facb6ac94699/photos/
Authorization: Bearer <JWT>
Content-Type: application/json
Accept: application/json
```

Listing 43: Odpowiedź HTTP — sukces

```
Content-Type: application/json
200 OK
Body:
{
  "listingId": 33db8939-5830-47b6-8e1b-facb6ac94699
  "photos": [
    "98606e9e-e43d-440c-8906-ee6dfc110198", "2c4fb159-1ede-424b-bd4a-bfd9e46f0894",
    "be515eb1-ed6d-4198-8be3-c8eca8a9983f"
  ]
}
```

Listing 44: Żądanie HTTP - Usunięcie danego zdjęcia

```
DELETE /api/v1/listings/33db8939-5830-47b6-8e1b-facb6ac94699/photos/be515eb1-ed6d
    -4198-8be3-c8eca8a9983f
Authorization: Bearer <JWT>
Content-Type: application/json
Accept: application/json
```

Listing 45: Odpowiedź HTTP — sukces

```
Content-Type: application/json
204 NO CONTENT
```

18 Wynajem pokoju

Niniejszy rozdział opisuje proces wynajmu pokoju przez użytkownika pełniącego rolę lokatora. Opis dotyczy wyłącznie logiki biznesowej realizowanej po stronie systemu. Mechanizm technicznej autoryzacji JWT został opisany w osobnym rozdziale i nie jest tutaj powielany.

Zakłada się, że żądanie dotarło do warstwy kontrolera z poprawnie ustawionym kontekstem użytkownika (identyfikator użytkownika oraz role).

18.1 Opis endpointu

- Metoda HTTP: `POST`
- Adres URL: `/api/v1/rentals`
- Endpoint chroniony: tak (JWT)

Dodatkowe endpointy procesu rezerwacji i płatności. Utworzenie „wynajmu” (`POST /rentals`) inicjuje obiekt `Booking`. Aby domknąć proces z diagramu przypadków użycia (akceptacja/odrzucenie, anulowanie, płatność), API udostępnia następujące operacje:

- `POST /api/v1/rentals/{rentalId}/accept` — akceptacja rezerwacji przez Właściciela (US-W4).
- `POST /api/v1/rentals/{rentalId}/reject` — odrzucenie rezerwacji przez Właściciela (US-W4).
- `POST /api/v1/rentals/{rentalId}/cancel` — anulowanie rezerwacji przez Lokatora (wariant z rysunku modułu wynajmu).
- `POST /api/v1/rentals/{rentalId}/pay` — inicjacja płatności (zwraca URL do bramki lub dane redirectu).
- `GET /api/v1/rentals/{rentalId}` — podgląd statusu rezerwacji (używane również przez integrację płatności do weryfikacji stanu).

18.2 Przykładowe żądanie

Listing 46: Żądanie HTTP — wynajem pokoju

```
POST /api/v1/rentals
Authorization: Bearer <JWT>
Content-Type: application/json
Accept: application/json
```

Listing 47: Ciało żądania — wynajem pokoju

```
{
  "roomId": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
  "startDate": "2024-11-01",
  "endDate": "2025-06-30"
}
```

18.3 Przykładowa odpowiedź — sukces

W przypadku gdy wskazany pokój jest dostępny w zadanym okresie oraz spełnione są warunki biznesowe, system tworzy nową rezerwację w statusie oczekującym na zatwierdzenie.

Listing 48: Odpowiedź HTTP — sukces

```
201 Created
Location: /api/v1/rentals/aa11bb22-cc33-dd44-ee55-ff6677889900
Content-Type: application/json
```

Listing 49: Ciało odpowiedzi — sukces

```
{
  "rentalId": "aa11bb22-cc33-dd44-ee55-ff6677889900",
  "status": "PENDING_APPROVAL",
  "createdAt": "2024-10-15T14:00:00Z",
  "totalPrice": 9600.00,
  "currency": "PLN",
  "resourceLink": "/api/v1/rentals/aa11bb22-cc33-dd44-ee55-ff6677889900"
}
```

18.4 Obsługa błędów

W trakcie realizacji procesu wynajmu mogą wystąpić następujące błędy:

- `400 Bad Request` – niepoprawny zakres dat,
- `403 Forbidden` – próba wynajmu własnego pokoju,
- `404 Not Found` – wskazany pokój nie istnieje,
- `409 Conflict` – pokój jest niedostępny w wybranym terminie.

Listing 50: Przykładowa odpowiedź — pokój zajęty

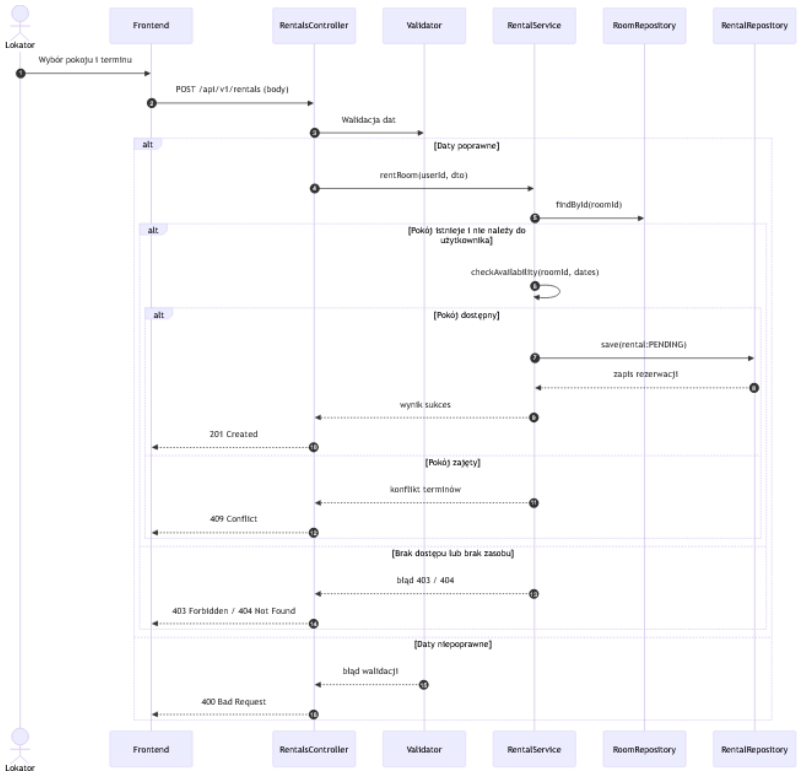
```
{
  "error": "RoomOccupied",
  "message": "Pokój jest niedostępny w wybranym terminie.",
  "unavailableDates": [
    { "from": "2024-12-01", "to": "2025-01-01" }
  ]
}
```

18.5 Diagram sekwencji wynajmu pokoju

Na rysunku 20 przedstawiono diagram sekwencji ilustrujący przebieg procesu wynajmu pokoju po stronie systemu.

18.6 Uwagi projektowe

Proces wynajmu uwzględnia walidację zakresu dat, weryfikację dostępności pokoju oraz kontrolę reguł biznesowych (m.in. zakaz wynajmu własnych zasobów). Zastosowanie statusu pośredniego (`PENDING_APPROVAL`) umożliwia dalszą obsługę procesu, np. akceptację przez wynajmującego.



Rysunek 20: Diagram sekwencji procesu wynajmu pokoju

Listing 51: Żądanie HTTP — utworzenie wątku

```
POST /api/v1/listings/c92f1c80-2d3a-4e5b-9a1f-8b2c3d4e5f6a/threads
Authorization: Bearer <JWT>
Accept: application/json
```

Listing 52: Odpowiedź — identyfikator wątku

```
{
  "threadId": "4f5d2f32-5c8f-4a1a-9e35-2e6f0d9b1a77",
  "listingId": "c92f1c80-2d3a-4e5b-9a1f-8b2c3d4e5f6a"
}
```

Uwagi komunikacyjne: pole `clientMessageId` pozwala klientowi na deduplikację w przypadku ponowień.

19 Moderacja i zgłoszenia naruszeń

Moduł moderacji opiera się o obiekt `ViolationReport`, który reprezentuje „sprawę” w panelu administracyjnym. Zgłoszenie może dotyczyć ogłoszenia lub użytkownika.

19.1 Zgłoszenie naruszenia przez użytkownika

- Metoda HTTP: `POST`
- Adres URL: `/api/v1/reports`
- Endpoint chroniony: tak (JWT)

Listing 53: Ciało żądania — zgłoszenie naruszenia

```
{
  "type": "LISTING",
  "targetId": "c92f1c80-2d3a-4e5b-9a1f-8b2c3d4e5f6a",
  "reason": "Podejrzenie oszustwa / niezgodne treści",
  "details": "W opisie znajdują się dane wrażliwe i linki zewnętrzne."
}
```

19.2 Panel Administratora — lista zgłoszeń

- Metoda HTTP: `GET`
- Adres URL: `/api/v1/admin/reports?page=0&size=20`
- Endpoint chroniony: tak (JWT + rola ADMIN)

Odpowiedź zawiera listę spraw wraz ze statusem (`Open/UnderReview/...`) oraz metadanymi, co umożliwia priorytetyzację.

19.3 Blokada użytkownika

- Metoda HTTP: `POST`
- Adres URL: `/api/v1/admin/users/{userId}/ban`
- Endpoint chroniony: tak (JWT + rola ADMIN)

Listing 54: Ciało żądania — blokada użytkownika

```
{
  "reason": "Powtarzające się naruszenia regulaminu"
}
```

Po skutecznej blokadzie system wykonuje sekwencję działań naprawczych:

1. Ustawia `User.status = BLOCKED`.
2. Ukrywa wszystkie aktywne ogłoszenia użytkownika (przejście w stan `HIDDEN_BY_MODERATION`).
3. Anuluje wszystkie przyszłe, opłacone rezerwacje (status `CONFIRMED`) dotyczące ogłoszeń tego użytkownika i automatycznie inicjuje proces zwrotu środków (Refund) w bramce płatności.
4. Rejestruje powiązanie tych akcji ze sprawą moderacyjną, aby zachować audytowalność decyzji.